



OPEN Urban low-altitude UAV path planning by fusing an enhanced A* algorithm with an adaptive artificial potential field method

Wei Gao^{1✉}, Linlin Li¹ & Dingying Pang²

To address the challenges posed by dense obstacles, complex dynamic avoidance, and poor path continuity in trajectory planning for multi-rotor UAVs operating in intricate urban low-altitude environments, this paper presents a multi-phase trajectory planning method (A*-APF) that integrates an enhanced A* algorithm with an adaptive artificial potential field (APF). The proposed approach supports the UAV's entire mission profile, including take-off, cruising, and landing. By constructing a 3D voxelized urban airspace model and introducing a phased advancement mechanism to establish sub-goals, together with strategies such as a dual-radius safety and obstacle avoidance model, dynamic step size adjustment, and 3D kinematic constraints, the heuristic function is optimized to enhance both path feasibility and safety. Furthermore, the method employs an improved APF combined force to guide heading direction and enables local obstacle avoidance corrections through the coupling of attractive and repulsive forces. The final path is globally and locally smoothed using cubic/quartic B-spline algorithms, ensuring continuous curvature and dynamic controllability. Validated through comprehensive comparative analyses and ablation studies, the simulation results quantitatively confirm that, compared with existing state-of-the-art improved algorithms, the proposed A*-APF method reduces average computation time by over 36% and path length by over 18%. Furthermore, the ablation experiments substantiate that the synergistic integration of the multi-phase mechanism and adaptive APF is critical for ensuring a 100% obstacle avoidance success rate and effectively responding to emergent threats.

Keywords Urban low-altitude, Trajectory planning, A* algorithm, Artificial potential field, Multi-stage modeling

Abbreviations

APF	Artificial potential field
UAM	Urban air mobility
UAV	Unmanned aerial vehicle
VTOL	Vertical take-off and landing
SAS	Sparse A* search
RRT	Rapidly-exploring random tree
BFS	Breadth-first search

Driven by the rise of the urban "low-altitude economy" and the concept of Urban Air Mobility (UAM), drones have demonstrated broad application prospects in fields such as urban logistics distribution, emergency response, and infrastructure inspection, creating a growing demand for their safe and efficient autonomous operation in urban low-altitude environments. Multi-rotor Unmanned Aerial Vehicles (UAVs), which do not require runways and are capable of Vertical Take-Off and Landing (VTOL), have become the primary choice for urban low-altitude transportation carriers. This is due to their advantages such as low operational costs, a simple structure that allows for easy maintenance, and a low lifecycle cost. However, the urban low-altitude environment poses significant challenges for path planning, characterized by dense obstacles, limited flight

¹College of Air Traffic Management, Civil Aviation University of China, Tianjin 300300, China. ²Civil Aviation Administration of China North China Regional Air Traffic Management Bureau, Beijing 100621, China. ✉email: atmgaao@126.com

altitudes, and frequent dynamic changes. These factors introduce high risks and uncertainties, necessitating robust capabilities in both global planning and local obstacle avoidance, which places higher demands on UAV trajectory planning [XX, XX^{1,2}].

Depending on the completeness of the environmental map, path planning methods can be categorized into two types: global path planning, which relies on prior knowledge of a complete map, and local path planning, which depends on real-time sensory information of the immediate environment [XX³]. Global path planning algorithms, which fall under static planning methods, can be broadly categorized into three types: sampling-based algorithms (e.g., RRT [XX⁴]), node search-based algorithms (e.g., A* [XX⁵] and Dijkstra [XX⁶]), and intelligent algorithms (e.g., genetic algorithm [XX⁷] and ant colony algorithm [XX⁸]). Local path planning algorithms, classified as a form of dynamic planning, comprise two main categories: curve fitting-based methods and numerical optimization-based techniques. Typical examples include Bézier curves [XX⁹] and approaches incorporating soft or hard constraints [XX¹⁰].

The conventional A* algorithm is widely used in global path planning due to its straightforward heuristic search mechanism and suitability for discrete spaces. However, in complex urban low-altitude environments, it faces challenges such as redundant node expansion, limited path smoothness, and insufficient flexibility in obstacle avoidance [XX¹¹]. The Artificial Potential Field (APF) method excels at local obstacle avoidance due to its high computational efficiency and rapid response, making it well-suited for dynamic obstacle avoidance and real-time path adjustments. However, it is prone to local minima and often struggles to ensure adherence to the global path [XX¹²]. In recent years, various studies have aimed to enhance the obstacle avoidance performance of the Artificial Potential Field (APF) method through techniques such as adaptive step size adjustment and the introduction of virtual targets, which have effectively improved path smoothness and increased obstacle avoidance success rates [XX¹³]. Additionally, hybrid strategies that integrate A* and APF have been proposed. These approaches employ A* for global path planning and APF for local path refinement, thereby combining global guidance with local adjustments to improve both path quality and robustness. As a result, these methods achieve a balance between path optimality and obstacle avoidance capability. In recent years, to address the stringent demands of urban environments, state-of-the-art trajectory planning approaches have rapidly evolved. Recent literature extensively explores optimization-based methods (e.g., minimum-snap and model predictive control) to ensure dynamic feasibility, as well as advanced heuristic-potential hybrids^{14–16} to balance global optimality with local reactivity. Multi-phase path planning techniques have also been introduced to handle distinct mission staging within urban models¹⁷. Furthermore, the idea of integrating APF into the planning process—particularly as an evaluation and cost mechanism to guide heuristic search—has shown great promise in advancing robotic navigation¹⁸.

However, a comprehensive review of these recent advancements reveals critical gaps. Most existing hybrid planners treat the mission as a single uniform search problem, lacking an adaptive mechanism to reconfigure spatial constraints (such as strict altitude layer preferences) dynamically across distinct flight phases. Furthermore, integration between reactive obstacle avoidance (APF) and discrete node expansion (A*) remains superficial; APF is often merely added as a static scalar cost penalty rather than being structurally utilized to filter kinematically infeasible expansions. Consequently, these state-of-the-art methods frequently suffer from redundant node exploration, poor physical continuity at phase transitions, and an over-reliance on conservative, non-dynamic step sizes.

Building upon the robust conceptual foundation of APF-guided search^{18,19}, our approach distinguishes itself by extending this integration into a dynamic, multi-phase 3D framework. Rather than employing a static cost function, our method dynamically reconfigures the APF constraints and heuristic weights according to specific flight phases, effectively bridging the gap between discrete node expansion and continuous local steering. This structural innovation directly resolves the lingering issues that significantly constrain computational efficiency and practical safety in dense, non-uniform urban low-altitude airspaces.

While hybrid A*-APF strategies and dynamic step-size adjustments have been explored in recent literature to address local minima and path smoothness^{20–22}, most existing approaches utilize APF primarily as a weighted term within the cost function or apply a uniform logic throughout the flight. In contrast, this paper proposes a system-level architectural innovation rather than a purely algorithmic one. The uniqueness of our method lies in the Phased State Machine framework, where the flight phases (take-off, cruise, and landing) act as distinct states that dynamically reconfigure the search space constraints. For instance, the altitude boundary constraint and layer-priority filtering are strictly enforced only during the “Cruise” state, whereas kinematic constraints are relaxed during transition states.

Building upon the aforementioned research, this paper proposes a multi-phase UAV trajectory planning method that integrates an improved A* algorithm with an adaptive artificial potential field (APF)—referred to as A*-APF—for use in complex urban low-altitude environments. The proposed method constructs a three-dimensional voxelized urban environment and incorporates a phase-adaptive advancement mechanism. It integrates a dual-radius safety and obstacle avoidance model with a dynamic step-size strategy, while embedding kinematic constraints. Local, real-time obstacle avoidance correction is achieved using an artificial potential field. Furthermore, the generated path is smoothed and optimized through cubic/quartic B-spline trajectory generation, ensuring feasibility, continuity, and dynamic controllability. This study aims to improve the safety, planning efficiency, and practical applicability of UAV trajectory planning in complex urban low-altitude environments, thereby providing a theoretical foundation and technical support for future autonomous UAV operations within urban air mobility systems.

Modeling of trajectory planning problem

To enhance flight safety and path optimization capabilities of UAVs in complex urban low-altitude environments, this paper establishes a trajectory planning framework with phased control logic that addresses the entire

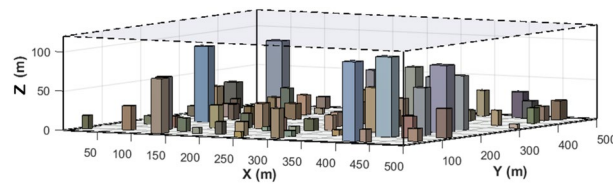


Fig. 1. Schematic diagram of urban obstacle modeling.

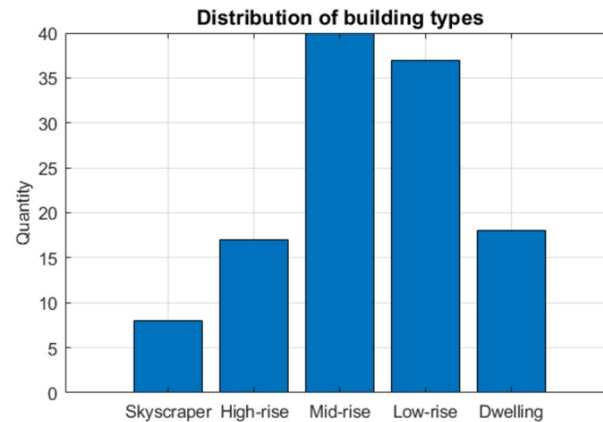


Fig. 2. Distribution diagram of urban building types.

mission profile, from takeoff and cruise to landing. Through the synergistic integration of three-dimensional voxelized airspace modeling, embedded kinematic constraints, and a phased sub-goal advancement mechanism, the complex and continuous urban low-altitude environment is transformed into a discrete, searchable, obstacle-avoidable, and controllable computational space. This approach provides a well-defined spatial framework and clear constraint boundaries for subsequent path searching and algorithm implementation. The mathematical modeling process underlying the trajectory planning problem will be systematically elaborated in the following section.

Planning space

A computable and discretized three-dimensional model of the urban low-altitude environment is constructed to accurately characterize obstacle distribution and delineate UAV-navigable airspace. The planning airspace refers to the designated region considered in UAV simulations. A three-dimensional Cartesian coordinate system is adopted to model this space, allowing any point within the planning area to be represented as (x, y, z) . Within this 3D domain, obstacles are modeled based on typical urban architectural features, as illustrated in Fig. 1. An example of obstacle distribution is provided in Fig. 2.

The planning airspace is defined as a three-dimensional Euclidean space $Q \subseteq R^3$. Given the input parameters (e.g., airspace dimensions), it is discretized into a voxel grid such that:

$$\begin{cases} x_i = x_{min} + i \cdot \delta, i = 0, 1, \dots, N_x - 1 \\ y_j = y_{min} + j \cdot \delta, j = 0, 1, \dots, N_y - 1 \\ z_k = z_{min} + k \cdot \delta, k = 0, 1, \dots, N_z - 1 \end{cases} \quad (1)$$

In Eq. (1), δ represents the grid resolution, $x_{min}, y_{min}, z_{min}$ denotes the minimum coordinate values of the airspace, Q specifies the spatial extent of the 3D airspace, and N_x, N_y, N_z indicates the number of voxels along the x-, y-, and z-axes, respectively.

Obstacle modeling (cuboid as an example):

$$B_m(p) = \max \left(\frac{|x - c_x|}{l_x/2}, \frac{|y - c_y|}{l_y/2}, \frac{|z - c_z|}{h/2} \right) \leq 1 \quad (2)$$

In Eq. (2), $B_m(p)$ denotes the implicit function of the m -th obstacle, (C_x, C_y, C_z) represents the coordinates of the obstacle center, (l_x, l_y, h) indicates the three-dimensional size of the obstacle, and $(l_x/2, l_y/2, h/2)$ specifies the center point of the voxel unit in three-dimensional space, which serves as the default reference point for UAV grid-based modeling.

Note that while obstacles are modeled as cuboids in the mathematical formulation for simplicity, our proposed algorithm utilizes a voxel-based representation in implementation. A voxel grid can approximate

arbitrary irregular shapes by discretizing the space. This capability is essential for realistic low-altitude path planning, where building geometries are diverse and unstructured.

Kinematic constraints

When conducting UAV trajectory planning, it is essential to consider two critical aspects: environmental factors and the UAV's own performance limitations. Environmental factors include the distribution of obstacles, while the UAV's intrinsic performance constraints—primarily related to flight dynamics—must be clearly defined to account for its physical maneuverability limits. If a planned trajectory exceeds the UAV's performance limits, it may result in hazardous situations such as loss of control or aerodynamic stall during flight.

Comprehensive consideration and appropriate constraint modeling of the aforementioned factors are essential for determining an optimal flight path from the initial position to the target destination, thereby ensuring both the safety and efficiency of UAV operations.

The position of the UAV in three-dimensional space is denoted as $P_n(x_i, y_i, z_i)$. The trajectory of the UAV can be represented by a sequence of nodes $\{S, P_1, P_2, \dots, P_{n-1}, G\}$, where S represents the start node, G the goal node, and $P_1, P_2, \dots, P_{n-1}$ are intermediate nodes. The kinematic constraints of the UAV are defined as follows:

1. The maximum yaw angle constraint is:

$$\psi = \tan^{-1} \left(\frac{y_{i+1} - y_i}{x_{i+1} - x_i} \right) \leq \psi_{max} \quad (3)$$

In Eq. (3), ψ denotes the real-time yaw angle, and ψ_{max} represents the maximum allowable yaw angle.

2. Maximum Pitch Angle Constraint is:

$$\theta = \tan^{-1} \left(\frac{z_{i+1} - z_i}{\sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}} \right) \leq \theta_{max} \quad (4)$$

In Eq. (4), θ represents the real-time pitch angle, and θ_{max} denotes the maximum allowable pitch angle.

3. Minimum Turning Radius Constraint is:

$$R_{min} = \frac{v^2}{g \cdot \tan(\varphi_{max})} \quad (5)$$

$$R_i \leq R_{max}, i = 1, 2, \dots, n \quad (6)$$

In Eqs. (5, 6), φ_{max} denotes the maximum bank angle, v represents the current flight velocity, g indicates the gravitational acceleration, R_i refers to the turning radius at the current node, and R_{min} specifies the minimum allowable turning radius.

4. Velocity and Acceleration Constraints is:

$$v_i = \frac{P_{i+1} - P_i}{\Delta t}, a_i = \frac{v_{i+1} - v_i}{\Delta t} \quad (7)$$

$$\|v_i\|_2 \leq v_{max}, \|a_i\|_2 \leq a_{max} \quad (8)$$

$$i = 1, 2, \dots, n - 1$$

5. Trajectory Distance Constraint is:

$$\sum_{i=1}^n \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2 + (z_{i+1} - z_i)^2} \leq L_{max} \quad (9)$$

In Eq. (7), L_{max} denotes the maximum allowable trajectory distance

Multi-phase Trajectory modeling

A phased trajectory planning model is established based on the unique characteristics and constraints of each mission phase—take-off, cruise, and landing—enabling precise control throughout the entire operation.

As illustrated in Fig. 3, to achieve phased trajectory planning for a UAV departing from the start point S , passing through transitional points A (initial altitude layer point, $P_A = (x_a, y_a, z_a)$) and B (point directly above the target, $P_B = (x_b, y_b, z_b)$), and finally landing at the goal point G ($P_G = (x_g, y_g, z_g)$), this paper introduces a phased subgoal progression mechanism into the conventional A* algorithm. This mechanism ensures that the trajectory strictly adheres to the "take-off—cruise—landing" flight procedure.

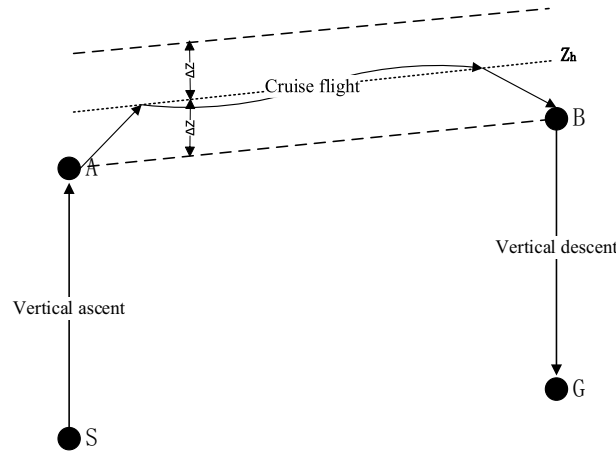


Fig. 3. Schematic diagram of multi-stage progression.

Phased progression mechanism

The flight phase is defined by an index $\phi \in \{1, 2, 3\}$, where $\phi = 1$ denotes the take-off phase (vertical ascent from S to A), $\phi = 2$ represents the cruise phase (level flight from A to B), and $\phi = 3$ indicates the landing phase (vertical descent from B to G).

The corresponding phased subgoal points are defined as follows:

$$P_{\text{goal}}^{(\phi)} = \begin{cases} P_A = (x_S, y_S, z_h - \Delta z), \phi = 1 \\ P_B = (x_G, y_G, z_h - \Delta z), \phi = 2 \\ P_G = (x_G, y_G, z_G), \phi = 3 \end{cases} \quad (10)$$

During the path search process, if the current node P_n satisfies the following condition:

$$\|P_n - P_{\text{goal}}^{(\phi)}\| < \epsilon_\phi \quad (11)$$

In Eq. (10,11), $P_{\text{goal}}^{(\phi)}$, represents the target point for each phase, ϵ_ϕ denotes the transition tolerance for each phase, which determines whether the current phase target is considered accomplished, thereby triggering an automatic transition to the next phase $\phi \rightarrow \phi + 1$. A schematic diagram of the multi-phase progression is illustrated in Fig. 3.

Flight phase modeling

Before detailing the continuous-time dynamics, it is imperative to clarify their role within the proposed discrete, voxel-based A* planning framework. The continuous dynamic equations (e.g., thrust models and PID-based altitude control laws) are not directly integrated into the discrete node expansion process. Instead, they serve two critical bridging functions: (1) defining the physically feasible flight envelope, which is subsequently used to derive the discrete kinematic constraints (e.g., maximum vertical step size and valid pitch/yaw transitions) enforced during the A* search; and (2) providing the underlying trajectory tracking logic that ensures the discrete waypoints generated by the multi-phase planner can be safely and smoothly executed by the UAV's low-level flight controller in continuous time (Fig. 4).

1. Take-off phase

During the take-off phase, the UAV ascends vertically from the starting point, rapidly and safely climbing to the recommended cruising altitude level.

To ensure that the thrust generated during the take-off phase is sufficient to overcome both gravitational and aerodynamic drag forces while maintaining vertical lift equilibrium—especially critical for multi-rotor configurations—the following thrust balance equation must be satisfied:

$$F \geq mg + \frac{1}{2}\rho_a v^2 SC_D + \frac{v^2}{R_{\min}} m \quad (12)$$

In Eq. (12), F denotes the total propeller thrust, m represents the mass of the UAV, ρ_a indicates the air density, S refers to the reference wing area, and C_D specifies the drag coefficient.

Kinematic propulsion is defined as:

$$\begin{aligned} z(t) &= z_s + v_z \cdot t + \frac{1}{2}a_z t^2 \\ x(t) &= x_s, y(t) = y_s \end{aligned}$$

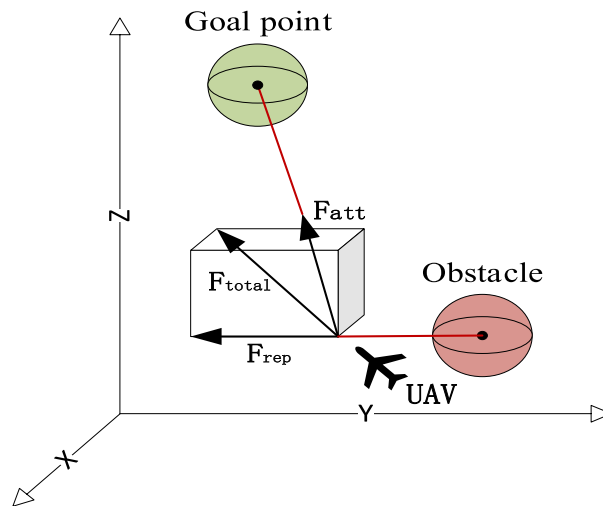


Fig. 4. Principle diagram of artificial potential field method.

$$\sigma_z = \frac{k \cdot \delta}{3}$$

$$z(t) = z_h - \Delta z + \sigma_z \quad (13)$$

In Eq. (13), σ_z denotes the altitude control range parameter, which is a scaling factor set according to the vertical voxel resolution and airspace height to normalize the penalty magnitude. z_h represents the recommended cruising altitude level, and Δz indicates the permissible altitude deviation range.

2. Cruise Phase

Upon reaching point A, the UAV enters the cruise phase, during which it climbs to the designated altitude level, optimizes its heading and velocity, and transitions as rapidly as possible into the prescribed cruising altitude. The kinematic propulsion is defined as follows:

$$\begin{cases} x(t) = x_s + v_x t \\ y(t) = y_s + v_y t \\ z(t) = z_a + v_z t \end{cases} \quad (14)$$

$$\mathbf{v} = v \begin{bmatrix} \cos\psi \cos\theta \\ \sin\psi \cos\theta \\ \sin\theta \end{bmatrix} \quad (15)$$

$$\tan\theta = \frac{|z_{i+1} - z_i|}{\sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}} \leq \tan\theta_{\max} \quad (16)$$

$$\Delta\psi = \tan^{-1} \left(\frac{y_{i+1} - y_i}{x_{i+1} - x_i} \right) \leq \Delta\psi_{\max} \quad (17)$$

$$z(t) \in [z_h - \Delta z, z_h + \Delta z]$$

To maintain stable flight at the current altitude level without abrupt transitions, a PID controller is incorporated to regulate the vertical velocity. This control strategy ensures that the UAV remains stabilized near the target altitude in dynamic environments, effectively resisting disturbances and minimizing tracking errors.

$$u_z = K_p(z_h - z) + K_d(\dot{z}_h - \dot{z}) + K_i \int (z_h - z) dt \quad (18)$$

$$K_p = 0.5 \cdot \frac{z_h}{\sigma_z}, K_d = 0.1 \cdot K_p, K_i = 0.01 \cdot K_p \quad (19)$$

In Eqs. (18,19), u_z denotes the altitude control command, while K_p , K_d , K_i and represent the proportional, derivative, and integral coefficients, respectively.

The UAV initiates the landing procedure when it reaches within a certain horizontal range of the destination point $P_G = (x_g, y_g, z_g)$. It then follows a gradual descent slope until reaching point $P_B = (x_b, y_b, z_b)$, located at a specified altitude directly above the target point. The descent trajectory is defined by the following equation:

$$\begin{cases} x(t) = x + v_x t \\ y(t) = y + v_y t \\ z(t) = z_b + v_z t, v_z < 0 \end{cases} \tag{20}$$

$$\mathbf{v} = v \begin{bmatrix} \cos \psi \cos \theta \\ \sin \psi \cos \theta \\ \sin \theta \end{bmatrix} \tag{21}$$

3. Landing Phase

At point *B*, located directly above the target point, the UAV initiates its final vertical descent from the current altitude in a smooth and controlled manner to ensure a safe landing. The vertical landing process begins as follows:

$$z(t) = z_b + v \cdot t + \frac{1}{2} a_z t^2 = z_g \tag{22}$$

$$x(t) = x_g, y(t) = y_g \tag{23}$$

and the horizontal velocity approaches zero: $v_x \approx 0, v_y \approx 0$, indicating that the UAV has entered the final vertical landing phase.

Algorithm principle

This paper enhances the conventional A* algorithm and the artificial potential field (APF) method, integrating them into a more effective hybrid approach by introducing a phased trajectory search strategy tailored for complex urban low-altitude environments. The proposed method incorporates APF-derived directional information into global path planning, combined with node expansion filtering mechanisms, dynamic step size control, and a dual-radius safety and obstacle avoidance model. This enables efficient path searching under limited computational resources while maximizing dynamic obstacle avoidance performance. The initial path is further refined and smoothed with B-spline curves, significantly improving trajectory flyability and curvature continuity. Comprehensive evaluations demonstrate that the proposed algorithm offers significant advantages in feasibility, safety, and computational efficiency, thereby providing a robust algorithmic foundation for subsequent simulation validation. To clearly distinguish the proposed architecture from conventional hybrid methods, a detailed comparison is presented in Table 1.

A* Algorithm

Within the framework of path planning algorithms, the traditional A* algorithm is a classic and widely adopted method. It typically utilizes a grid-based representation to process the planning space. After rasterizing the environment, the A* algorithm searches for a path using a predefined cost function, which is one of its core components. This function integrates multiple factors to evaluate the cumulative cost from the starting point to the current point, as well as to the goal. Guided by this cost function, the algorithm systematically explores the grid-based planning space, gradually identifying a trajectory with minimal cost that serves as a feasible path for unmanned systems or other mobile agents.

The algorithm utilizes environmental data as heuristic information to guide the search, thereby reducing both the search space and computational time. Central to the A* algorithm are the OPEN and CLOSE lists. The OPEN list contains nodes that have been discovered but not yet expanded, along with their associated cost information, whereas the CLOSE list retains nodes that have already been expanded. The algorithm operates iteratively by selecting the node with the lowest cost from the OPEN list for expansion, moving it to the CLOSE list, and adding its successor nodes to the OPEN list. The cost function used in the A* algorithm is defined as follows:

$$f(n) = g(n) + h(n) \tag{24}$$

Feature	Traditional A*-APF / Improved A* [16–20, Refs]	Proposed A*-APF architecture
Role of APF	Cost Integration Only: APF is typically added as a scalar value to the heuristic function H(n) or cost G(n)	Dual Role (Vector+Cost): Functions as both a cost term and a force vector for local steering correction during node expansion
Task Phase Logic	Static Waypoints: Intermediate points serve only as geometric sub-goals; the algorithm's internal logic remains constant	Finite State Machine: The "Phased Advancement Mechanism" acts as an finite state machine. Constraints like U_{bound} and heuristic weights dynamically switch based on the phase index ϕ
Node Expansion	Standard Grid Search: Uses fixed 26-connectivity or random sampling (e.g., RRT-based hybrids)	Kinematic Pruning: Integrates "Field of View" Directional Filtering and Altitude Layer Priority to prune kinematically infeasible nodes
Step Size Strategy	Distance-Based: Step size often depends solely on the distance to obstacles	Dual-Radius Coupled: Step size is strictly governed by the Dual-Radius Safety Model (R_{safe} , R_{avoid}), ensuring kinematic feasibility before expansion

Table 1. Comparison of algorithmic architectures and key mechanisms between the proposed method and existing strategies.

In Eq. (24), n denotes the node to be expanded. $g(n)$ represents the actual cost from the start position to the current node n ; $h(n)$ indicates the heuristic estimated cost from node n to the goal; and $f(n)$ corresponds to the total cost value of node n .

To better adapt the path planning model to multi-phase flight missions in complex urban low-altitude environments, this paper introduces phase-specific optimizations and enhancements to the conventional A* algorithm. The global path planning process aims to minimize a comprehensive cost function by expanding the node with the lowest total cost $f(n)$. In each phase ϕ , the A* algorithm employs the following heuristic cost function:

$$f^{(\phi)}(n) = g(n) + w_1 \cdot h^{(\phi)}(n) + w_2 \cdot U_{rep}(n) + w_3 \cdot U_{height}^{(\phi)}(n) + w_4 \cdot U_{bound}^{(\phi)}(n) \quad (25)$$

$$w_1 + w_2 + w_3 + w_4 = 1 \quad (26)$$

In Eqs. (25, 26), $f^{(\phi)}(n)$ denotes the total cost of node n in phase ϕ , serving as a primary criterion for optimal path selection within the planning algorithm. $g(n)$ represents the actual cost, quantified as the cumulative path cost from the phase-specific starting point to node n . $h^{(\phi)}(n)$ corresponds to the Euclidean distance between the current node P_n and the designated target point of phase ϕ ($P_{goal}^{(\phi)}$). $U_{rep}(n)$ denotes the repulsive potential cost term induced by obstacles, representing the obstacle avoidance cost at the current position, the APF is applied as a scalar potential energy field added to the cost function to represent the environmental threat level, rather than a force vector; $U_{height}^{(\phi)}(n)$ refers to the altitude preference penalty term, which prevents deviation from the regulated altitude range and is active only during the cruise phase; $U_{bound}^{(\phi)}(n)$ indicates the flight altitude boundary penalty term, which is also exclusively effective in the cruise phase.

The altitude preference penalty term is defined as a soft constraint according to the following expression:

$$U_{height}^{(2)}(n) = w_h \cdot [\max(0, |z_n - z_h| - \sigma_z)]^2 \quad (27)$$

Altitude Boundary Hard Constraint (Applicable only in phase $\phi = 2$ (cruise phase)):

$$U_{bound}^{(\phi)}(n) = \begin{cases} 0, & z_h - \Delta z \leq z(n) \leq z_h + \Delta z \\ \infty, & \text{other} \end{cases} \quad (28)$$

The switching logic relies on the permissible altitude deviation range, Eq. 27 applies a penalty cost scaled by σ_z . Soft Constraint: When deviation $|z_n - z_h| \leq \Delta z$, Hard Constraint: When deviation $|z_n - z_h| > \Delta z$, Eq. 28 triggers, and the node is marked invalid.

The subgoal point $P_g(n)$ is dynamically updated according to the current flight phase.

$$P_{goal}^{(\phi)} = \begin{cases} P_A = (x_S, y_S, z_h - \Delta z), & \phi = 1 (\text{take-off}) \\ P_B = (x_G, y_G, z_h - \Delta z), & \phi = 2 (\text{cruise}) \\ P_G = (x_G, y_G, z_G), & \phi = 3 (\text{landing}) \end{cases} \quad (29)$$

In Eq. (29), w_h represents the unified penalty strength coefficient.

Artificial potential field (APF) method

While the conventional Artificial Potential Field (APF) method is well-established in the literature, it fundamentally suffers from local minima and numerical instability near obstacles. To address this within our hybrid framework, we rigorously delineate the roles of scalar cost evaluation and vector force steering.

During the global A* node expansion, the APF obstacle threat is simplified into a scalar cost penalty term added to the heuristic function (as seen in Section "A* Algorithm"). However, for local trajectory correction within the dynamic step size control, the APF generates a vector force ($\vec{F}_{total}$) used exclusively for directional steering. To resolve the numerical instability typically caused by infinite repulsive potentials when the distance to an obstacle approaches zero, we implement a dual-protection mechanism: (1) Pre-check Rejection, where nodes within the safety radius (R_{safe}) are pruned before force computation; and (2) Force Clamping, where a maximum force threshold F_{max} is established to cap the magnitude of the repulsive vector. The refined mathematical formulation of our localized APF application is defined as follows:

$$U_{att}(X) = \frac{1}{2} K_{att} \rho(X, X_{goal})^2 \quad (30)$$

$$U_{rep}(\vec{P}) = \begin{cases} \frac{1}{2} K_{rep} \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right)^2, & \rho \leq \rho_0 \\ 0, & \rho > \rho_0 \end{cases} \quad (31)$$

$$\rho(X, X_{obs}) \leq \rho_0 \quad (32)$$

In the equation, U_{att} and U_{rep} denote the attractive and repulsive potential fields, respectively; K_{att} and K_{rep} represent the attraction and repulsion gain coefficients; ρ indicates the relative distance between the UAV and the target point; $\rho(X, X_{obs})$ corresponds to the Euclidean distance to the nearest obstacle; and ρ_0 specifies the influence radius.

To enable path exploration in a three-dimensional grid environment while dynamically responding to obstacle threats, this paper defines two types of spatial volumes around the UAV: a safe space and an avoidance space. As illustrated in Fig. 5, the UAV is modeled as a point mass. Both spaces are spherical in shape: the safe space, depicted as a red sphere, has a radius of R_{safe} , while the avoidance space, represented by a yellow sphere, has a radius of R_{avoid} . The safe space ensures collision-free operation, and the avoidance space triggers the avoidance control mechanism.

In this study, the attractive potential field is generated by the target point of each phase, guiding the UAV toward the designated goal. The magnitude of the attractive potential is proportional to the distance between the UAV and the target. The attractive field is defined as the negative gradient of this potential:

$$U_{att}(n) = \frac{1}{2} k_{att} \cdot \|\vec{P}_n - \vec{P}_{goal}^{(\phi)}\|^2 \quad (33)$$

$$\vec{F}_{att}(n) = -\nabla U_{att}(n) = k_{att} \cdot \left(\vec{P}_{goal}^{(\phi)} - \vec{P}_n \right) \quad (34)$$

The repulsive potential field dynamically adjusts the trajectory within the obstacle avoidance radius to maximize the safety distance between the UAV and obstacles, thereby preventing collisions. Generated by obstacles, this field repels the UAV away from them. The repulsive potential is effective only within the avoidance radius; it becomes infinite within the safety radius to strictly enforce collision avoidance. The repulsive force field is defined as the negative gradient of this potential:

$$U_{rep}(n) = \begin{cases} \frac{1}{2} k_{rep} \left(\frac{1}{D_n} - \frac{1}{R_{avoid}} \right)^2, & R_{safe} < D_n \leq R_{avoid} \\ \infty, & D_n \leq R_{safe} \\ 0, & D_n > R_{avoid} \end{cases} \quad (35)$$

$$\vec{F}_{rep}(n) = -\nabla U_{rep}(p) = K_{rep} \left(\frac{1}{D_n} - \frac{1}{R_{avoid}} \right) \cdot \frac{\vec{P}_n - \vec{P}_o}{D_n^3} \quad (36)$$

In the equation, D_n denotes the distance between the UAV and the obstacle, $\vec{F}_{att}(n)$ represents the attractive force field, and $\vec{F}_{rep}(n)$ corresponds to the repulsive force field. Unlike the scalar cost used in global planning, here the APF generates a gradient force vector used for local steering and dynamic obstacle avoidance.

In implementation, to prevent numerical instability when the distance D_n approaches R_{safe} , we apply two strategies: Force Saturation: A maximum force threshold F_{max} is set. Any calculated force exceeding this limit is clamped to F_{max} . Pre-check: Before calculation, any node with $D_n \leq R_{safe}$ is identified as an obstacle node and discarded directly.

$$F_{max} = \|\vec{F}_{rep}(n) (D_n = R_{safe} + \delta)\| \quad (37)$$

These measures ensure the algorithm remains stable without numerical overflow."

To guide the search direction toward the target while effectively avoiding obstacles, this paper incorporates the resultant artificial potential field force, $\vec{F}_{total}$ into the node expansion phase of the A* algorithm as a

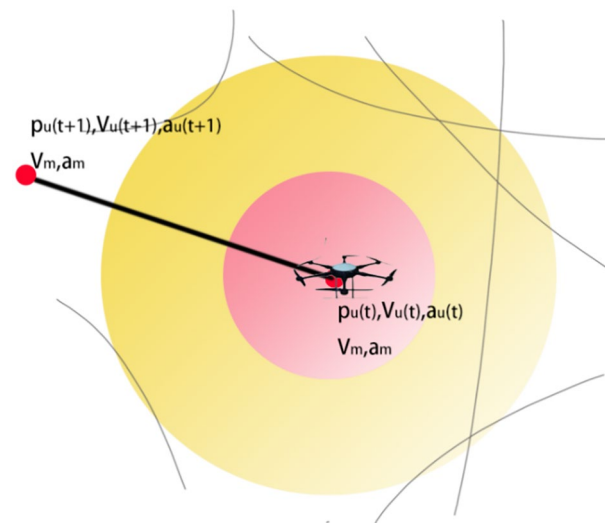


Fig. 5. Schematic diagram of safety space and avoidance space.

criterion for node selection. The total force acting on the UAV is defined as the vector sum of the attractive force and all repulsive forces:

$$\vec{F}_{total} = \vec{F}_{att} + \sum_i \vec{F}_{rep,i} \quad (38)$$

The final trajectory:

$$\mathcal{P} = \mathcal{P}^{(1)} \cup \mathcal{P}^{(2)} \cup \mathcal{P}^{(3)} \quad (39)$$

Each path segment is generated by minimizing the heuristic cost function $f^{(\phi)}(n)$ to drive the search process, where the segment is defined as:

$$\mathcal{P}^{(\phi)} = \arg \min_{\text{path from } P_{start}^{(\phi)} \text{ to } P_{goal}^{(\phi)}} \sum_n f^{(\phi)}(n) \quad (40)$$

In the equation, $\vec{F}_{total}$ denotes the resultant force in the artificial potential field method, and $\mathcal{P}$ represents the path segment for each phase.

The flowchart illustrating the principle of the proposed algorithm is shown in Fig. 6:

Node expansion

In this paper, the 'Enhanced A*' architecture integrates multiple spatial and kinematic filters to optimize node expansion within the 3D voxel grid. To resolve potential conflicts among these filtering mechanisms and clarify the integration of kinematic constraints, the node expansion evaluates candidate neighbors according to a strict hierarchical execution sequence. Constraints are explicitly classified into 'Hard Constraints' (used for absolute pruning) and 'Soft Penalties' (used for cost evaluation):

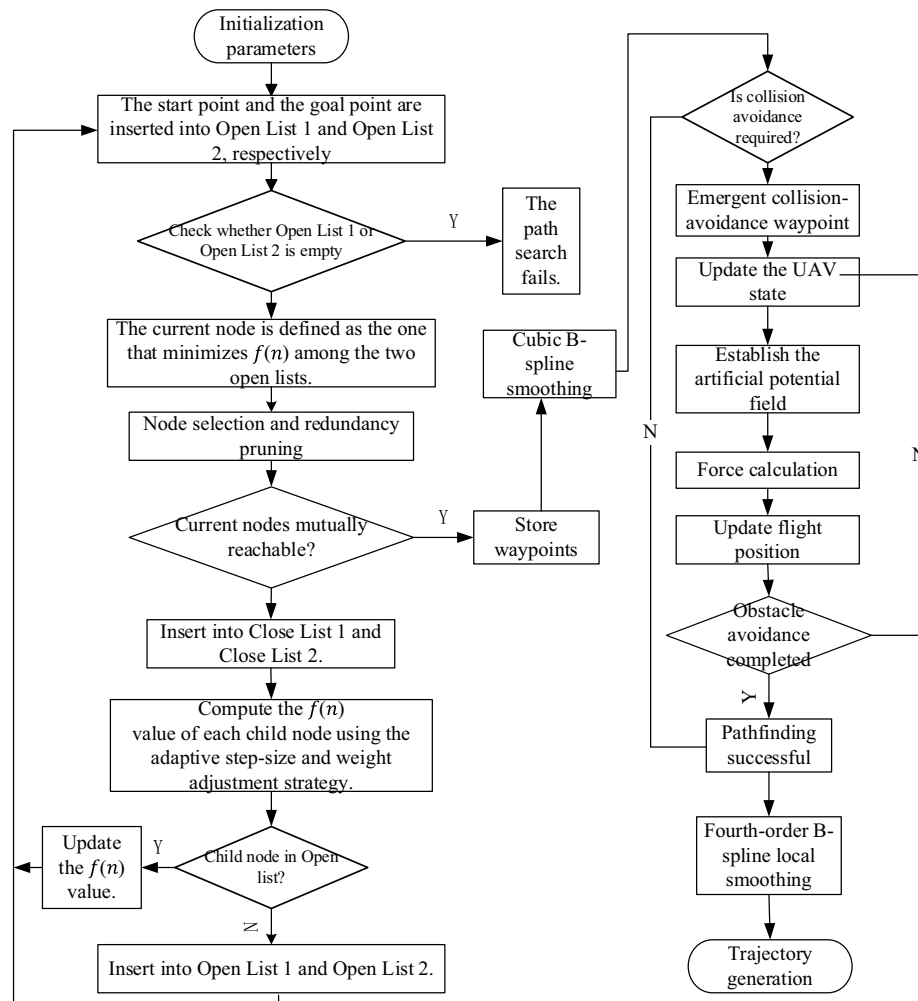


Fig. 6. Algorithm flowchart.

1. Kinematic Pruning (Hard Constraint): As defined in Section "Kinematic constraints", limits on maximum yaw, pitch, and turning radius are treated as strict physical boundaries. Any candidate neighbor whose required displacement vector violates these limits is immediately pruned from the search tree.
2. Safety Space Pre-check (Hard Constraint): To prevent numerical instability and ensure absolute collision avoidance, the distance D_n to the nearest obstacle is calculated. Any node where $D_n \leq R_{safe}$ is definitively rejected before any force calculation.
3. Directional Consistency Filtering (Soft/Heuristic Filter): Valid nodes are then evaluated using the 'Field of View' cone (detailed later). This mechanism heuristically encourages forward progression towards the current sub-goal.
4. APF-based Force Correction (Conditional Priority): This step is activated exclusively when a node falls within the Avoidance Zone ($R_{safe} < D_n \leq R_{avoid}$). If a conflict arises between the directional consistency filter (pushing toward the target) and the APF safety compliance (pushing away from an obstacle), avoiding the obstacle takes absolute priority. Candidate directions that oppose the APF safety vector are discarded to guarantee survival.

If all candidate nodes in the OPEN list are exhausted without reaching the target due to severe environmental blockage or kinematic limits, the system triggers a 'Failure Logic', halting path progression to avoid infinite loops.

1. Safety Zone Division and Dynamic Step Size Control

Based on the distance between the UAV and obstacles, regions with different threat levels are defined as detailed below. The exploration step size is dynamically adjusted accordingly to balance search efficiency and obstacle avoidance precision.

- Safety Zone:

$$D_n \geq R_{avoid}$$

Under this condition, where the current position is sufficiently distant from obstacles and lies within the safety zone, the maximum step size l_{max} and maximum velocity are applied for node expansion to facilitate rapid exploration.

- Obstacle Avoidance Zone:

$$R_{safe} < D_n \leq R_{avoid}$$

The current node is located in a potential threat zone. To ensure safe obstacle avoidance, the minimum step size l_{min} is adopted, combined with a correction strategy based on the resultant direction of the Artificial Potential Field (APF), enabling precise trajectory adjustment and guidance.

- Danger Zone:

$$D_n \leq R_{safe}$$

In this zone, where the UAV is in extreme proximity to obstacles and faces a high risk of collision, the strategy is to immediately halt expansion and initiate a hovering maneuver, requiring no further path correction. Where D_n denotes the distance between the UAV and the obstacle, l_{min} represents the minimum step size, and l_{max} indicates the maximum step size.

2. Three-Dimensional Neighborhood Expansion and Directional Filtering:

In a three-dimensional grid environment, each node can theoretically generate child nodes in 26 neighboring directions, encompassing vertical, horizontal, and diagonal movements. To mitigate inefficient search, enhance computational efficiency, and improve path quality, the following filtering strategies are implemented:

Altitude Layer Priority Filtering: From the full set of 26 possible neighboring nodes generated from the current node (representing the fundamental neighborhood), those with heights closest to the recommended cruising altitude level are prioritized. This approach effectively narrows down the candidate set.

$$\mathcal{N} = \{n_{n+1} \in \mathcal{N}_{26} \mid |z_{n+1} - z_h| \leq \Delta z\} \quad (41)$$

$$z_h * 0.2 = \Delta z \quad (42)$$

In Eqs. (40, 41), $\mathcal{N}$ denotes the set of prioritized nodes, z_{n+1} represents the altitude of the candidate child node, z_h indicates the recommended cruising altitude level, and Δz specifies the permissible altitude deviation range.

The gravitational vector pointing from the current node toward the target (point A, B, or G) is calculated. Only expansions aligned with the target direction are retained. This first-round filtering of all candidate expansions evaluates whether the current expansion direction advances toward the target, based on the cosine of the angle between the expansion direction and the target direction. The cosine value is computed for each candidate directional vector $\vec{F}_{dir}$ as follows:

$$\vec{F}_{dir}^{(i)} = \frac{\vec{P}_{n+1}^{(i)} - \vec{P}_n}{\|\vec{P}_{n+1}^{(i)} - \vec{P}_n\|} \quad (43)$$

$$\vec{F}_{goal} = \frac{\vec{P}_{goal}^\phi - \vec{P}_n}{\|\vec{P}_{goal}^\phi - \vec{P}_n\|} \quad (44)$$

$$\cos\theta = \frac{\vec{F}_{goal} \cdot \vec{F}_{dir}}{\|\vec{F}_{goal}\| \|\vec{F}_{dir}\|} \geq \eta \quad (45)$$

In the equation, $\vec{F}_{dir}^{(i)}$ denotes the candidate direction vector, $\vec{F}_{goal}$ represents the target direction vector, and η serves as the Directional Consistency Threshold.

This strategy effectively eliminates reverse, lateral, and other expansion directions that are detrimental to phased progression, thereby ensuring that the path planning process maintains clear directional guidance while preserving safety. Geometrically, η defines a 'Field of View' cone towards the target. Only child nodes falling within this cone are considered valid. This prevents the search tree from expanding backwards or excessively laterally, strictly enforcing the forward progression logic of the multi-phase mechanism and pruning inefficient search branches.

3. APF Repulsive Field Direction Correction

During filtering within the obstacle avoidance zone, the current expansion direction is evaluated to determine whether it simultaneously facilitates obstacle avoidance and progression toward the target. This process selects optimal directions from multiple candidates—ensuring obstacle avoidance without deviating from the primary course. Directions deemed conducive to both approaching the target and avoiding obstacles are retained; otherwise, they are discarded.

To eliminate ambiguity between the scalar cost evaluation and vector force steering mechanisms of the APF, a strict conditional boundary is established using a spatial indicator function $\mathbb{I}_{avoid}(P_n)$, defined by the distance to the nearest obstacle D_n :

$$\mathbb{I}_{avoid}(P_n) = \begin{cases} 1, & R_{safe} < D_n \leq R_{avoid} \\ 0, & D_n > R_{avoid} \end{cases} \quad (46)$$

When $\mathbb{I}_{avoid}(P_n) = 0$, the UAV operates in the safe zone, and the APF functions exclusively as a scalar penalty term $U_{rep}(n)$ within the A* heuristic cost function (Eq. 25). Conversely, when $\mathbb{I}_{avoid}(P_n) = 1$, the avoidance mechanism is triggered, and the APF strictly acts as a directional vector constraint. To guarantee collision avoidance, the candidate displacement vector d must align with the total APF resultant force $\vec{F}_{total}$. This compliance is enforced via the dot-product inequality:

$$\frac{d \cdot \vec{F}_{total}}{\|d\| \|\vec{F}_{total}\|} \geq \cos(\xi) \quad (47)$$

where ξ is the safety compliance threshold. Candidate nodes failing to satisfy this inner product constraint directly oppose the physics-based safety field and are explicitly discarded.

$$\cos(\phi) = \frac{\vec{F}_{dir} \cdot \vec{F}_{total}}{\|\vec{F}_{dir}\| \|\vec{F}_{total}\|} \geq \xi \quad (48)$$

In Eq. (48), ξ represents the threshold parameter for the artificial potential field method. This parameter acts as a 'Safety Compliance Filter'. It evaluates whether a candidate move 'agrees' with the physics-based safety force $\vec{F}_{total}$. A value greater than ξ implies the candidate node effectively moves the UAV away from danger, whereas a value below ξ indicates a movement that opposes the safety force, which is immediately discarded to prevent collisions.

4. Hierarchical Expansion and Kinematic Constraint Pruning

To further reduce inefficient search operations and ensure that the generated path satisfies the kinematic constraints of the UAV, a hierarchical expansion strategy is adopted. This approach preselects key nodes in both the horizontal and vertical directions that comply with the aforementioned constraints, thereby minimizing subsequent redundant searches while adhering to velocity and acceleration limits.

$$\begin{cases} \|\vec{v}_{n+1}\| \leq v_{max} \\ \|\vec{a}_{n+1}\| \leq a_{max} \\ |\Delta\psi| \leq \psi_{max} \\ |\Delta\theta| \leq \theta_{max} \end{cases} \quad (49)$$

Horizontal Expansion. The change in heading angle induced by each candidate node is calculated. Within each layer, valid headings are selected either uniformly across the allowable angular range or according to a predefined strategy.

Vertical Expansion. The pitch angle of each candidate node is computed. Valid pitch angles are selected per layer, either uniformly within the permissible angular range or based on a specific strategy.

The kinematic constraints (Eqs. 3–6) are strictly enforced via a neighbor pruning strategy. During node expansion for the current node P_n :

To rigorously enforce the continuous kinematic limits within the discrete 3D voxel grid, the required yaw, pitch, and turning radius for each candidate neighbor must be algebraically quantified prior to node expansion. Let the current node be $P_n(x_n, y_n, z_n)$ and the candidate expansion node be $P_{n+1}(x_{n+1}, y_{n+1}, z_{n+1})$. Defining the discrete displacement vector as $d = P_{n+1} - P_n$, the resulting discrete yaw angle ψ_{n+1} and pitch angle θ_{n+1} are computed as:

$$\psi_{n+1} = \arctan 2(y_{n+1} - y_n, x_{n+1} - x_n) \quad (50)$$

$$\theta_{n+1} = \arcsin \left(\frac{z_{n+1} - z_n}{\|d\|} \right) \quad (51)$$

Furthermore, the discrete turning radius $R_{discrete}$ between consecutive segments is geometrically approximated by:

$$R_{discrete} = \frac{\|d\|}{2 \sin(|\psi_{n+1} - \psi_n|/2)} \quad (52)$$

These discrete variables act as a strict Boolean filter. A candidate node is appended to the OPEN list if and only if it satisfies the hardware-constrained boundaries: $|\psi_{n+1} - \psi_n| \leq \psi_{max}$, $|\theta_{n+1} - \theta_n| \leq \theta_{max}$, and $R_{discrete} \geq R_{min}$. Candidates violating any of these inequalities are immediately pruned to guarantee physical feasibility.

If any value exceeds the limits (ψ_{max} or θ_{max}), the candidate is deemed unreachable and is discarded immediately (i.e., not added to the OPEN list).

This ensures the generated path strictly adheres to the UAV's physical limits. Kinematic constraints are enforced as hard constraints by immediately pruning neighbor nodes that violate limits calculated via displacement vectors, whereas altitude preference is applied as a soft constraint in the cost function.

5. Emergency Hover Trigger Conditions

As the ultimate safety barrier, the system immediately halts path expansion and initiates a hover maneuver upon detection of any collision risk. If the distance $D_{(n+1)}$ of any candidate child node satisfies $D_{(n+1)} \leq R_{safe}$, it is determined that an obstacle has intruded into the safety radius. The current path expansion process is immediately terminated, and an emergency hover command is issued. This check is performed after obstacle pruning and the final kinematic screening, serving as a fail-safe mechanism.

Phase transition constraints and failure logic

To ensure the robustness of the multi-phase mechanism, we define the strict boundary conditions for phase switching, failure determination, and exception handling as follows:

1. **Transition Tolerance Constraint** The transition tolerance ϵ_ϕ (Eq. 11) determines the capture region of the sub-goal $P_{goal}^{(\phi)}$. To prevent the search path from oscillating around the target due to grid discretization, ϵ_ϕ must satisfy the step-size constraint defined in Table 3:

$$\epsilon_\phi > l_{max}$$

In this study, given $l_{max} = 1.5$ m, we set $\epsilon_\phi = 2.0$ m. This ensures that the capture condition $\|P_n - P_{goal}^{(\phi)}\| \leq \epsilon_\phi$ is physically achievable.

2. Logical Definition of Failure

If the sub-goal is completely enclosed by obstacles, the A* algorithm will explore all reachable nodes. The mission is mathematically determined as a Failure if and only if the OPEN list is exhausted without reaching the target:

$$\text{State} = \text{Failure} \Leftrightarrow (OPEN = \emptyset) \wedge (\|P_{best} - P_{goal}^{(\phi)}\| > \epsilon_\phi)$$

Under this condition, the system terminates the planning task to avoid infinite loops.

3. Non-Regressive Phase Logic

To guarantee mission efficiency, the phase index ϕ follows a monotonically non-decreasing logic ($\phi_{k+1} \geq \phi_k$). Regression to a previous phase is prohibited. If the UAV encounters a dynamic threat where forward progression is blocked, it triggers the Emergency Hover mechanism (Section "Node expansion") rather than regressing:

Action = Hover if ∃ Obstacle s.t. $D_{n+1} \leq R_{safe}$

This ensures the UAV maintains a safe holding state until the path clears or a local replanning is initiated.

4. Engineering Fallback and Replanning Policy

To ensure practical deployability and robustness in real-world scenarios, an explicit engineering closed-loop fallback mechanism is defined for typical failure cases:

- Global Search Failure: If the OPEN list is completely exhausted due to excessively dense obstacles or over-constrained kinematic pruning, the UAV immediately triggers an ‘Emergency Hover’ state. The system halts forward progression and alerts the ground station or upper-level mission planner for a global rerouting command.
- Post-Smoothing Safety Failure: If the generated continuous B-spline trajectory fails the convex-hull collision check (as detailed in Section “B-Spline Trajectory Smoothing”), the fallback action reverts the trajectory to the validated discrete waypoints. The planner then executes a localized refinement by inserting intermediate nodes to iteratively tighten the curve until the segment passes the safety certification.

This explicit termination and fallback logic ensures the vehicle remains in a safe, mathematically verified state under all edge cases.

B-spline trajectory smoothing

To enhance path smoothness and dynamic flyability, this paper employs cubic and quartic B-spline curves to perform segmental trajectory fitting and curvature optimization. Specifically, Cubic B-spline is applied for global path smoothing to ensure basic continuity, while Quartic B-spline is employed for local trajectory correction to achieve higher-order smoothness required for dynamic obstacle avoidance.

Unlike global optimization methods or Bézier curves, B-splines were selected for their superior local control property, which allows APF-based local corrections without distorting the entire global trajectory. To ensure this smoothing is physically valid in a full three-dimensional workspace, the control points are defined as 3D spatial coordinates $P_i(x_i, y_i, z_i)$. A critical challenge in 3D trajectory smoothing—particularly in urban environments—is the vertical (Z-axis) deformation. To prevent the smoothing process from generating paths that exceed the UAV’s vertical performance limits, the Z-axis manipulation is strictly bounded by the multi-rotor’s maximum climb and descent rates.

To balance computational efficiency with these 3D multi-rotor dynamics, we implement a hybrid strategy: Cubic B-splines (C2) are used for the static global path to ensure general 3D acceleration continuity. In contrast, Quartic B-splines (C3) are strictly applied during local dynamic obstacle avoidance to guarantee continuous Jerk in all three axes, thereby preventing control oscillations during rapid spatial maneuvers without violating the enforced vertical kinematic boundaries. A theoretical comparison justifying this selection is presented in Table 2.

$$B_{i,0}(t) = \begin{cases} 1, & t_i \leq t \leq t_{i+1} \\ 0, & \text{else} \end{cases} \tag{53}$$

$$B_{i,k}(t) = \frac{t - t_i}{t_{i+k} - t_i} B_{i,k-1}(t) + \frac{t_{i+k+1} - t}{t_{i+k+1} - t_{i+1}} B_{i+1,k-1}(t) \tag{54}$$

$$\begin{cases} B_{0,3}(t) = \frac{1}{6}(1-t)^3 \\ B_{1,3}(t) = \frac{1}{6}(3t^3 - 6t^2 + 4) \\ B_{2,3}(t) = \frac{1}{6}(-3t^3 + 3t^2 + 3t + 1) \\ B_{3,3}(t) = \frac{1}{6}t^3 \end{cases} \tag{55}$$

$$P(t) = \sum_{i=0}^3 P_i B_{i,3}(t), t \in [0, 1] \tag{56}$$

Curvature Calculation Formula:

$$\kappa(t) = \frac{\| \mathbf{P}'(t) \times \mathbf{P}''(t) \|}{\| \mathbf{P}'(t) \|^3} \tag{57}$$

Feature	Bézier curves	Minimum-Snap Opt	Proposed B-splines
Local Control	Low (Global effect)	N/A (Global Optimization)	High (Local modification)
Computation	Low	High (Quadratic Programming)	Low (Analytical)
Continuity	Variable	High (C ⁴)	Tunable (C ² /C ³)
Jerk Support	Possible	Yes	Yes

Table 2. Comparison of smoothing techniques for trajectory planning.

$$\kappa_{max} \leq \frac{1}{R_{min}} \quad (58)$$

To achieve the smoothest possible path under the maximum curvature constraint, the control points must satisfy the following optimization objective:

$$\min_{\{P_i\}} \max_t |\kappa(t)| \quad (59)$$

In the equation, $P(i)$ denotes the curve equation corresponding to the i -th control point, and $B_{i,k(t)}$ represents the basis function of the k -th order B-spline.

Given the control points P_p , a series of curve segments of the cubic B-spline function can be derived using the above equation, resulting in a cubic B-spline curve as illustrated in the Fig. 7.

When an obstacle enters the avoidance radius of the UAV, the resultant force from the APF is applied for trajectory correction. To ensure seamless integration of the adjusted trajectory with the global path, quartic B-spline smoothing is employed. The steps for quartic smoothing are as follows:

- Path Discretization

$\{P_0, P_1, \dots, P_N\}$ Extract Key Points from the Adjusted Path to facilitate subsequent curve fitting.

- High-Order Curve Fitting

$$P(t) = \sum_{i=0}^{N+3} B_i^4(t) \cdot Q_i \quad (60)$$

Quartic B-spline fitting is applied to the path to enhance its smoothness.

While B-spline fitting guarantees mathematical continuity, the geometric curvature $\kappa(t)$ of the smoothed 3D trajectory must be strictly bounded by the multi-rotor UAV's dynamic actuation limits to ensure flight feasibility. During cruise velocity v_{cruise} , the lateral centripetal acceleration a_{lat} required to track the curve is dictated by the maximum bank angle ϕ_{max} . This fundamental aerodynamic relationship establishes an absolute upper bound on the permissible curvature:

$$a_{lat} = v_{cruise}^2 \kappa(t) \leq g \tan(\phi_{max}) \quad (61)$$

$$\kappa(t) \leq \frac{g \tan(\phi_{max})}{v_{cruise}^2} \quad (62)$$

Consequently, this physical limit is enforced as a hard constraint during the subsequent curvature optimization process. If the evaluated curvature $\kappa(t)$ at any discrete interval exceeds this threshold, the algorithm iteratively injects intermediate control points to relax the local spline geometry, thereby mathematically preventing actuator saturation and aerodynamic stall during complex 3D maneuvers.

- Curvature Optimization

$$\min \sum_{t=0}^1 \left(\kappa(t) - \frac{1}{R_{min}} \right)^2 \quad (63)$$

The control points are adjusted to bring the path curvature close to the theoretical maneuvering limit, thereby satisfying the kinematic constraints of the vehicle.

- Tracking Accuracy Constraint

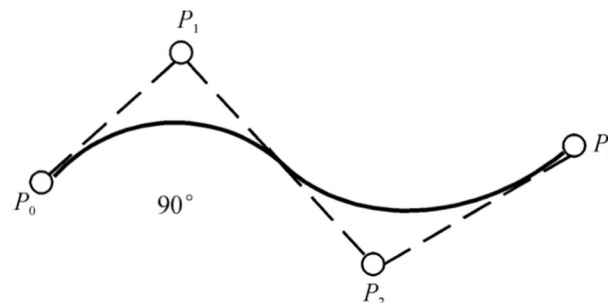


Fig. 7. Schematic diagram of cubic B-spline.

$$P(t_k) - Q_{k2} \leq \epsilon \quad (64)$$

The optimized path must remain aligned with the original obstacle avoidance trajectory to ensure navigation precision.

In the equation, $P(t)$ denotes the path position at the curve parameter t ; Q_i represents the corresponding control point; $P(t_k)$ indicates the position of the fitted curve at a specific parameter value t_k ; Q_k refers to a discrete point on the original path; and ϵ specifies the maximum allowable deviation tolerance.

While the discrete waypoints generated by the A*-APF planner are guaranteed to be collision-free, the continuous curve segments produced by B-spline smoothing may theoretically deviate and intersect with obstacle boundaries during cornering. To ensure rigorous safety and practical deployability, we implement a post-smoothing continuous collision check leveraging the convex-hull property of B-splines.

Following the safe-corridor certification concepts proposed in recent literature, the generated trajectory is strictly constrained such that the convex hull of the control points for each curve segment remains entirely within the obstacle-free voxel space. If a posteriori continuous collision check detects that a smoothed segment violates the minimum safety margin, the algorithm triggers a localized refinement: intermediate control points are iteratively inserted to tighten the B-spline convex hull to the original discrete path until the continuous corridor is mathematically certified as collision-free.

Experiments and simulations

To validate the feasibility of the proposed integrated algorithm in complex urban low-altitude environments, we conducted a comparative simulation of UAV trajectory planning using multiple algorithms in a three-dimensional scenario featuring both static and dynamic obstacles, representative of typical urban aerial operations. The experiments were designed with varying obstacle densities, task complexities, and flight conditions. A systematic comparison was conducted between the proposed algorithm and conventional methods (SAS, APF, RRT*) as well as state-of-the-art improved A*-APF algorithms to demonstrate its superiority in complex environments.* Furthermore, a comprehensive ablation study was performed to quantitatively verify the individual contributions of key modules—such as the multi-phase mechanism and node expansion strategy—to the overall performance. The simulation environment was constructed using a discretized 3D voxel map, incorporating typical takeoff and landing point configurations, obstacle distributions, and scenarios involving emergent obstacles. Performance was evaluated using multiple metrics, including path length, computation time (referring to the execution time for path planning, not the UAV flight duration), node utilization efficiency, and obstacle avoidance success rate. A comprehensive comparative analysis was conducted to evaluate the performance of the proposed algorithm in terms of pathfinding efficiency, obstacle avoidance capability, and adaptability to dynamic flight conditions, thereby providing reliable quantitative support for its application in urban low-altitude environments. In this simulation, the obstacle density (Ω) is mathematically defined as the ratio of the number of grid cells occupied by obstacles (N_{obs}) to the total number of grid cells (N_{total}) in the planning space. It is calculated as:

$$\Omega = \frac{N_{obs}}{N_{total}} * 100\% \quad (65)$$

To ensure the reproducibility of the simulation results and the rigorosity of the 50 independent trials, a standardized random map generation process was implemented based on this density definition. For each trial k (where $k = 1, 2, \dots, 50$), a specific integer random seed was fixed to initialize the pseudo-random number generator, ensuring that the obstacle distribution for each density level remains consistent across different algorithm tests. The generation process proceeds as follows: First, the 3D airspace is initialized as an empty voxel grid, where the target number of occupied voxels N_{obs} is calculated based on the specified density Ω using Eq. (58). Subsequently, the center coordinates (C_x, C_y, C_z) of each obstacle are generated using a uniform distribution within the planning boundaries $[0, X_{max}] \times [0, Y_{max}] \times [0, Z_{max}]$. Finally, to simulate the diversity of urban buildings, the dimensions (l_x, l_y, h) of each obstacle are randomized within a predefined range (e.g., length and width $\in [5, 20]$ m, height $\in [10, 100]$ m). Obstacles are iteratively added to the map until the actual obstacle density reaches the target Ω , with overlapping obstacles allowed to form complex, irregular building clusters.

While the fundamental algorithmic benchmarking in this section utilizes randomly generated axis-aligned cuboids to strictly control the obstacle density parameter (Ω), real urban environments are vastly more complex. It is crucial to emphasize that the proposed algorithm does not rely on the geometric properties of cuboids. Because the planning space is uniformly discretized into a 3D voxel grid (as defined in Section "Planning Space"), any irregularly shaped structure or non-axis-aligned building is naturally represented as a discrete cluster of occupied voxels. To explicitly demonstrate the algorithm's capability to handle complex, non-cuboidal geometries, a subsequent experiment (detailed in the latter part of Section "3D Simulation") applies the algorithm to real-world geographic data (OpenStreetMap) of Shanghai Lujiazui, successfully navigating a dense cluster of highly irregular skyscrapers.

Simulations were performed on a computer running Windows 11 (64-bit), equipped with an Intel® Core™ i7-9750H CPU at 2.6 GHz and 16 GB of RAM, using MATLAB 2023a as the simulation platform. The relevant simulation parameters are summarized in Table 3.

The selection of key experimental parameters (e.g., map dimensions, obstacle density, and algorithmic weights) was rigorously established to reflect realistic urban UAV operational constraints. The map sizes (e.g., 100×100 m for 2D benchmarking and larger 500×500 m areas for 3D tests) were chosen to represent typical delivery sectors and building block scales in modern micro-airspace logistics. The obstacle densities (Ω) varying up to 35% simulate extreme high-density urban clusters, ensuring the algorithms are tested under near-worst-case collision scenarios.

Parameter	Symbol	Value
Safety Radius /m	R_{safe}	2
Avoidance Radius /m	R_{avoid}	4
Minimum Step Size /m	l_{min}	0.5
Maximum Step Size /m	l_{max}	1.5
Transition Tolerance /m	ϵ_{φ}	2.0
Maximum Yaw Angle	ψ_{max}	$\pm 45^{\circ}$
Maximum Pitch Angle	θ_{max}	$\pm 30^{\circ}$
Maximum Bank Angle	ϕ_{max}	$\pm 30^{\circ}$
Distance Weight	w_1	0.4
Safety Weight	w_2	0.3
Altitude Boundary Weight	w_3	0.1
Altitude Layer Convergence Weight	w_4	0.2
Attractive Gain Coefficient	K_{att}	0.8
Repulsive Gain Coefficient	K_{rep}	1.2
Altitude Control Parameter	σ_z	10
Directional Consistency Threshold	η	0.7
APF Safety Threshold	ξ	0.2

Table 3. Parameter initialization value table.

Furthermore, the heuristic weight coefficients (w_1, w_2, w_3, w_4) and APF gain parameters (K_{att}, K_{rep}) were not chosen arbitrarily. They were determined through a systematic grid-search sensitivity analysis conducted during preliminary experiments. We iteratively tested various combinations and evaluated the Pareto frontier based on three primary metrics: obstacle avoidance success rate, computational search time, and global path length deviation. The selected values in Table 3 represent the empirical Pareto-optimal configuration that strictly prioritizes flight safety while minimizing computational overhead in dense 3D voxel environments.

2D simulation

Before evaluating the algorithm in fully complex 3D urban environments, initial simulations were conducted in a simplified 2D setting. The inclusion of these 2D experiments is not to simulate actual operational flight, but to provide a controlled, decoupled baseline for fundamental algorithmic validation. In a 2D grid, the core search efficiency, node expansion logic, and path deviation rate (optimality) can be rigorously benchmarked against standard algorithms (like BFS and simple APF) without being obscured by the highly coupled effects of 3D kinematic constraints, pitch/yaw limits, and altitude layer preferences. This foundational validation ensures that the core A*-APF fusion logic is mathematically sound and efficient before extending it to the more restrictive three-dimensional domain. The shortest flight distance obtained by the Breadth-First Search (BFS) algorithm was used as the benchmark for path planning. The improved A*-APF algorithm was compared against the Artificial Potential Field (APF) algorithm, the Rapidly-exploring Random Tree (RRT) algorithm, and the SAS algorithm. Under simplified two-dimensional constraints, simulations were conducted on a 100×100 grid map with six obstacle density settings: 10%, 15%, 20%, 25%, 30%, and 35%. For each density, 100 simulations were performed. The experimental results demonstrate the search performance of the four algorithms, with averaged outcomes compared in terms of path accuracy and computational time, as shown in Fig. 8.

The search time required by our method and other compared methods increases with rising obstacle density. The results indicate that in randomized maps, the average execution time of all methods grows as the number of obstacles increases. Among the four algorithms, the A*-APF algorithm outperforms the others in terms of search time. It effectively reduces search time in environments with low obstacle density. Under high obstacle density conditions, such as at 35% obstacle ratio, the average execution time of A*-APF is reduced by 50% compared to the original SAS algorithm, demonstrating its superior performance.

To quantify the optimality of the planned trajectory, we define the Path Deviation Rate (δ). It represents the percentage difference between the path length generated by the algorithm (L_{algo}) and the theoretical shortest path length (L_{opt}) obtained by the Benchmark algorithm (BFS). A lower deviation rate indicates a path that is closer to the global optimum. The formula is defined as:

$$\delta = \frac{L_{algo} - L_{opt}}{L_{opt}} * 100\% \tag{66}$$

The deviation rate of the path length obtained by the proposed method and other compared methods increases with rising obstacle density. By varying the number of obstacles in randomized maps, Fig. 9 illustrates the deviation rates of the path length for the four methods. As shown, RRT performs poorly in larger-scale maps, while the SAS algorithm maintains a deviation rate within 10%. Among them, the vector cross-product optimized APF algorithm is effective but relatively sacrifices accuracy. When the obstacle density reaches 20%,

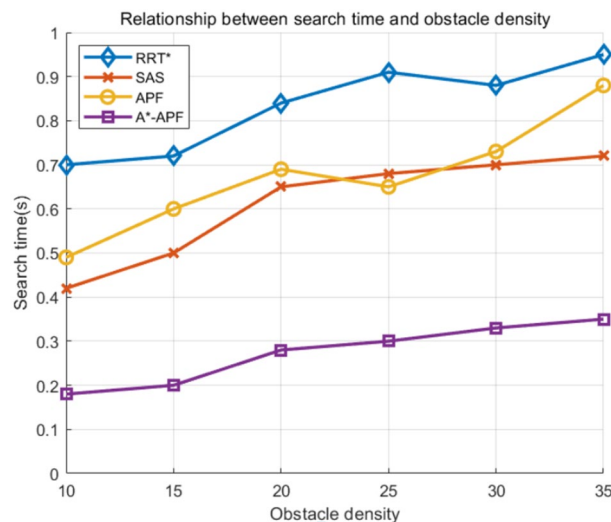


Fig. 8. Comparison of search time versus obstacle density.

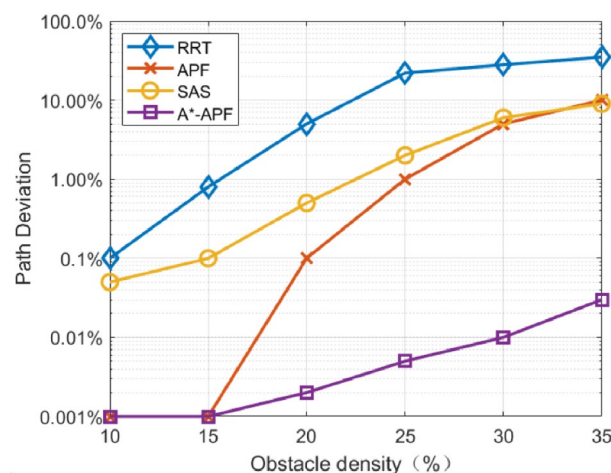


Fig. 9. Relationship diagram between path deviation rate and obstacle density.

the deviation of our method is only 0.0072%, ensuring that the searched path is relatively optimal. Moreover, as the obstacle density increases, the proposed algorithm consistently maintains a low deviation rate.

3D simulation

According to the simulation setup, experiments were conducted in a 3D simplified low-altitude airspace map. The top and side views of the static trajectory planning results for the four algorithms are shown in Fig. 10. Each algorithm was tested 50 times. To visualize the algorithmic stability, Fig. 11 illustrates the specific computation time for each of the 50 independent trials (raw data), rather than an average value. The blocks in the diagram denote simulated buildings. Buildings whose height surpasses the altitude of the start and end points are colored red. The simulation results are recorded in Table 4.

As shown in the figure, the UAV departs from the origin and successfully avoids static obstacles using the traditional SAS, APF, RRT, and the proposed A*-APF algorithms, each planning a feasible trajectory from the start to the goal point. Combined with the data in the table, it is evident that the path generated by the A*-APF algorithm proposed in this paper significantly outperforms the other methods in terms of search time, number of explored nodes, and path length. The traditional SAS algorithm and the improved A*-APF algorithm both found the same path; however, the path search time required by the traditional SAS algorithm increased by 1 second compared to the improved method. The A*-APF algorithm, applied to UAV trajectory planning, effectively reduces search time and path length while enhancing overall search efficiency. This is achieved through dynamic adjustment of heuristic function weights and an adaptive step size strategy, all while satisfying the platform's inherent constraints. The comparative simulation results presented above demonstrate the effectiveness and advancement of the proposed A*-APF algorithm.

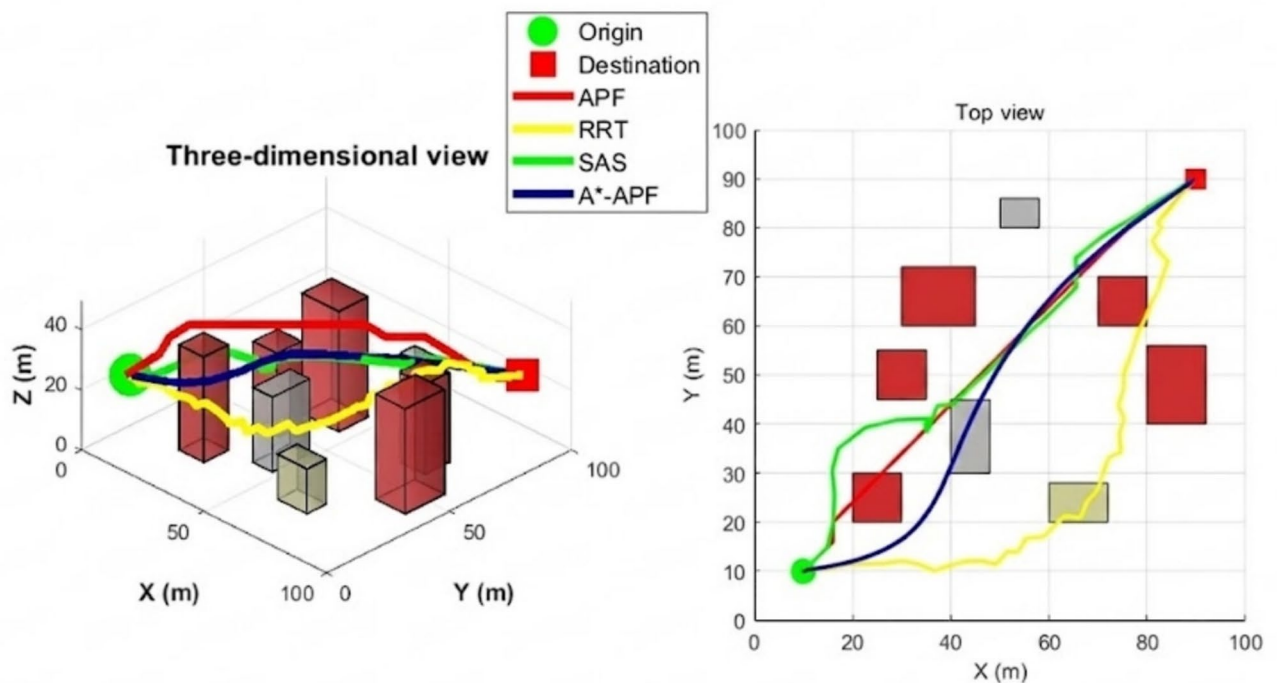


Fig. 10. Relationship diagram between search time and deviation rate.

According to the simulation configuration, experiments were conducted in a 3D complex low-altitude airspace environment. The top and side views of the static trajectories planned by the four algorithms are shown in Fig. 12, with buildings exceeding 80 m in height highlighted in red. Each algorithm was executed 50 times. The simulation results are documented in Table 5.

As illustrated in the figure, all algorithms successfully planned a trajectory from the start to the goal point in the complex low-altitude airspace. Combined with the data presented in the table, it is evident that the path generated by the A*-APF algorithm remains significantly superior to those of other methods in terms of search time, number of explored nodes, and path length. The comparative simulation results presented above validate the effectiveness and advancement of the improved algorithm proposed in this paper for urban low-altitude applications.

To demonstrate the algorithm's capability in handling complex, non-Manhattan geometries, we conducted a simulation using real-world geographic data of Shanghai Lujiazui, extracted from OpenStreetMap (OSM). This area represents low-altitude flight environments due to its dense cluster of skyscrapers with irregular footprints (e.g., the circular Shanghai Tower and the tapered Jin Mao Tower).

As shown in Fig. 13, the proposed planner successfully navigates this complex voxelized environment. The simulation validates that our method is not limited to the simplified cuboid obstacles defined in Section "Planning Space". By leveraging the voxel grid, the algorithm accurately identifies safe corridors between irregular buildings. The resulting trajectory (green curve) exhibits a complete UAV mission profile: a vertical ascent to 50m, a smooth B-spline cruise avoiding various building shapes, and a precision vertical descent, confirming the method's robustness in real-world urban applications.

Comparative analysis with existing improved A*-APF algorithms

To further demonstrate the superiority of the proposed method over state-of-the-art hybrid planning strategies, this section conducts a comparative evaluation against two established improved A*-APF algorithms cited in the literature: the method proposed by Tang et al.¹⁵ and the method proposed by Guo et al.¹⁶. Unlike the basic baselines (SAS, APF, RRT), these reference methods also integrate heuristic search with potential field guidance to address complex environments.

To ensure a rigorous comparison, the algorithms from Ref.¹⁵ and Ref.¹⁶ were reproduced locally and tested within the identical simulation framework (35% obstacle density) described in Table 2. The data presented are statistical averages derived from 50 independent trials, ensuring the significance of the results.

Comparison with reference¹⁵

Figure 14 illustrates the trajectory comparison between the proposed A*-APF algorithm and the improved A*-APF method from Ref.¹⁵. Both algorithms successfully generated collision-free paths in the dense building cluster. However, the trajectory generated by the method in Ref.¹⁵ exhibits unnecessary detours when navigating around the central high-rise obstacles, resulting in a longer flight distance. This mechanism effectively filters out redundant expansions in irrelevant altitude layers, allowing the UAV to identify a more direct and efficient path

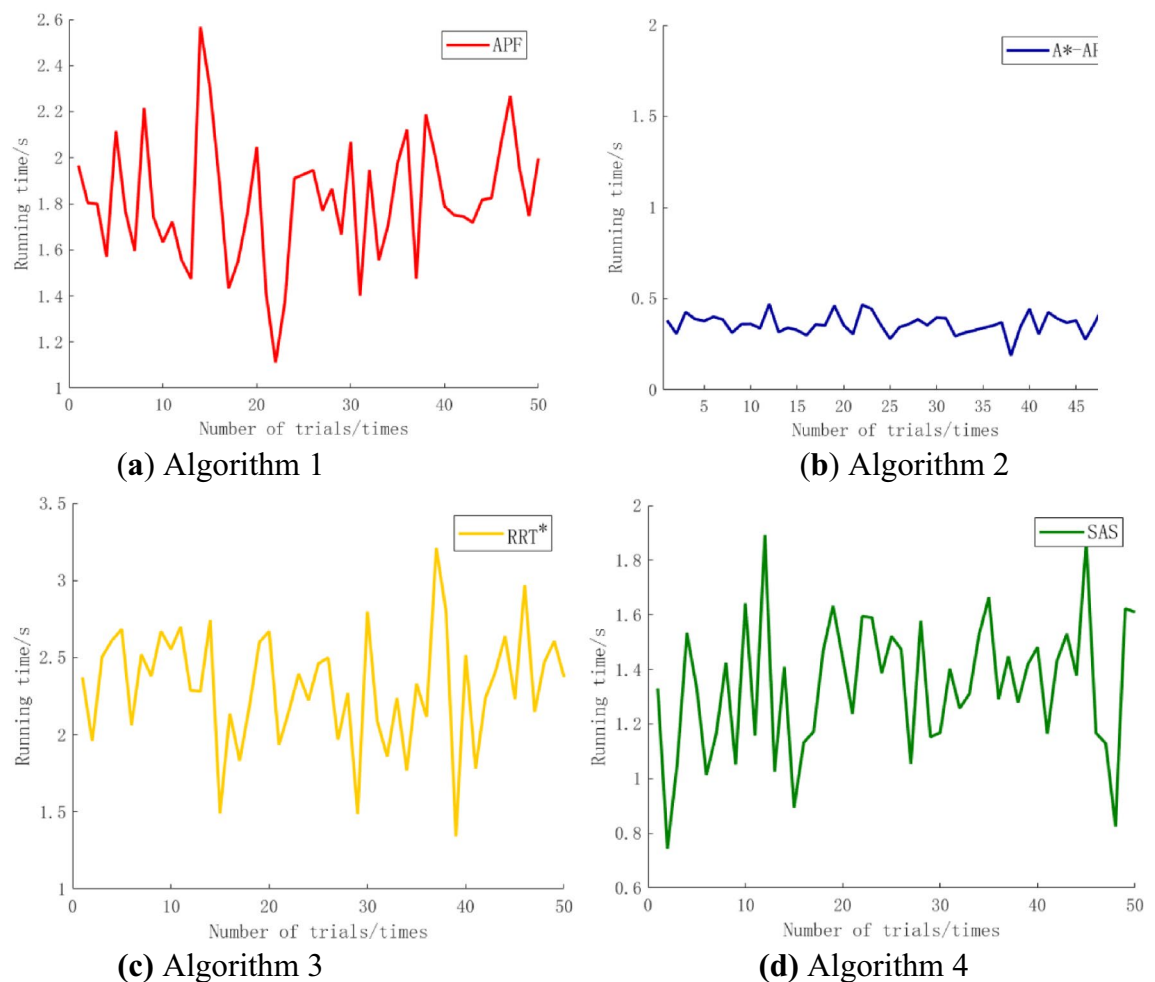


Fig. 11. Line chart of computation time for different algorithms.

Algorithm	Time/s	Number of searched nodes	Number of valid nodes	Flight Distance /m
SAS	1.35	1554	198	202.5
RRT	2.34	2027	217	229.3
APF	1.81	1892	205	161.6
A*-APF	0.37	1154	132	145.1

Table 4. Table of simulation result records.

through the obstacle gaps. Note that buildings taller than the flight altitude are highlighted in red to indicate collision risks.

Comparison with reference¹⁶

Figure 15 presents the comparison with the algorithm from Ref.¹⁶. While the method in Ref.¹⁶ improves upon traditional A* by incorporating repulsive potential fields to guide the search away from obstacles, it lacks a dynamic step-size adjustment strategy. Consequently, it requires a higher density of search nodes to maintain safety near obstacles, leading to increased computational overhead. As observed in the figure, the proposed algorithm employs a Dual-Radius Safety Model combined with Dynamic Step Size, enabling large steps in open spaces and fine-grained adjustments near threats. This results in a smoother trajectory with significantly fewer search nodes. Note that buildings taller than the flight altitude are highlighted in red to indicate collision risks.

Quantitative performance analysis

The statistical average results for computation time, number of expanded nodes, and path length are summarized in Table 6.

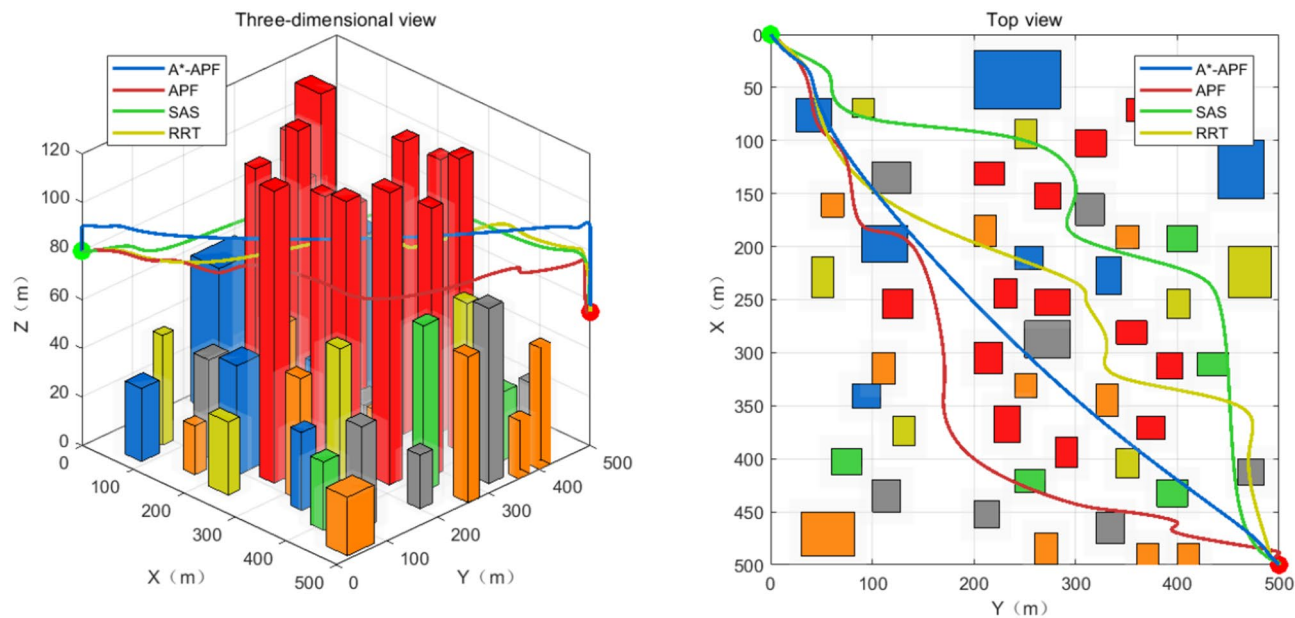


Fig. 12. Complex low-altitude airspace trajectory planning diagram.

Algorithm	time/s	Number of searched nodes	Number of valid nodes	Flight Distance /m
SAS	6.29	5854	398	1392.5
RRT	7.73	6527	417	1229.3
APF	6.01	6192	405	1271.6
A*-APF	1.09	3825	342	735.1

Table 5. Table of simulation result records.

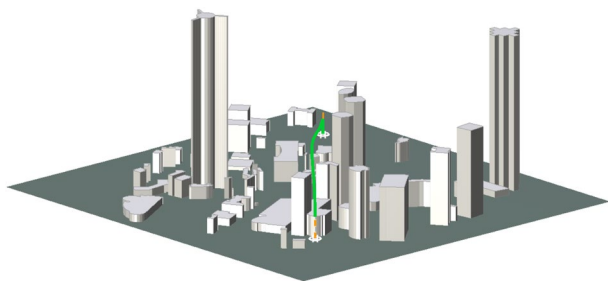


Fig. 13. 3D Real-World Urban Environment Simulation.

As indicated in Table 6, The proposed method reduces computation time by 46.2%. This is mechanistically caused by the Phased Advancement Mechanism. Unlike the global search in Ref.¹⁵, our method acts as a state machine that prunes irrelevant altitude layers (e.g., restricting nodes to the cruise layer during phase $\Phi=2$), effectively eliminating redundant computations². The number of valid nodes is reduced by 33%. This is attributed to the Dual-Radius Dynamic Step Size. Unlike Ref.¹⁶ which relies on conservative steps for safety, our finite-state logic enforces maximum step sizes in the “Safe Zone” ($D_n > R_{avoid}$) for rapid convergence, and switches to fine-grained steps only when necessary.

These results verify that the proposed method is not only superior to traditional baselines but also outperforms existing state-of-the-art hybrid algorithms in terms of planning speed and trajectory quality in complex urban environments.

Ablation study

To systematically quantify the individual contributions of each module within the proposed framework and verify their adaptability to complex cityscapes, a comprehensive step-by-step ablation study was conducted.

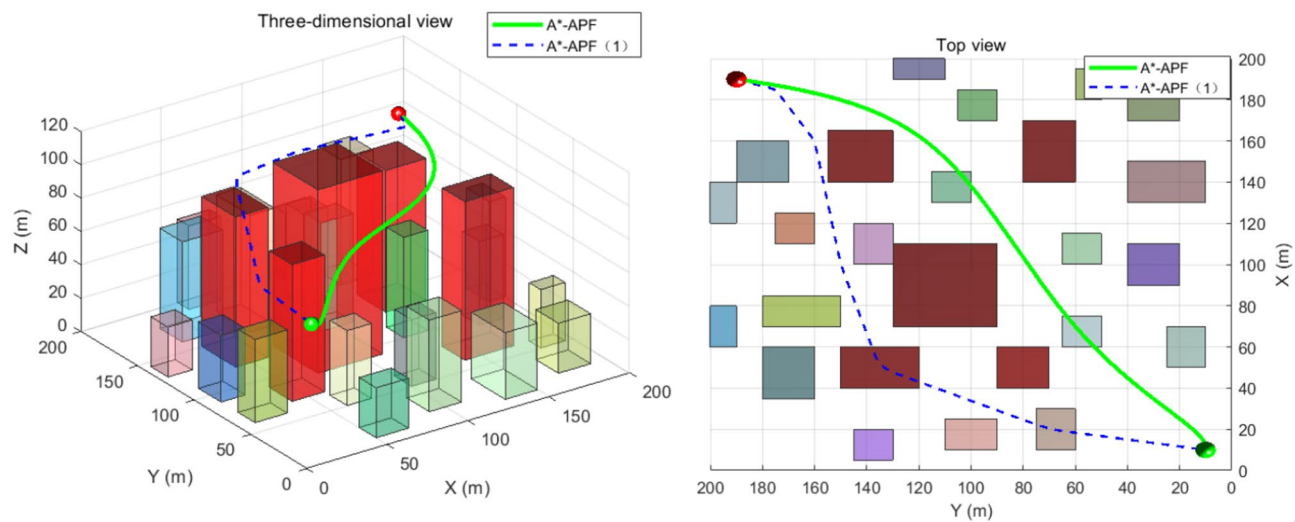


Fig. 14. Trajectory comparison between A*-APF and A*-APF(1) in Ref.¹⁵ in a high-density obstacle environment.

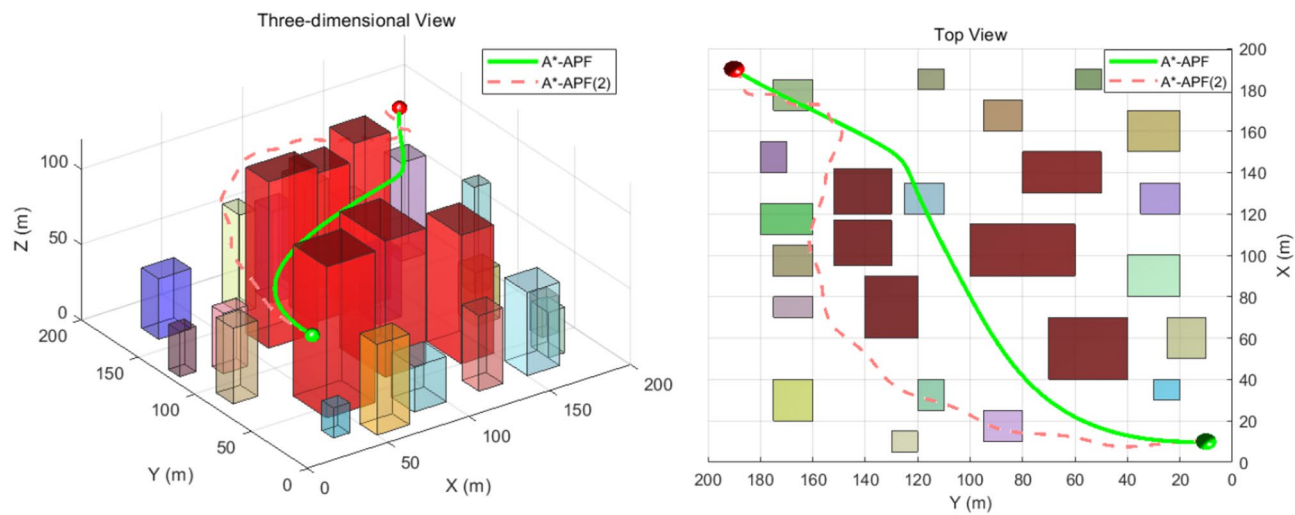


Fig. 15. Trajectory comparison between A*-APF and A*-APF(2) in Ref.¹⁶ in a high-density obstacle environment.

Algorithm	Time/s	Number of searched nodes	Number of valid nodes	Flight Distance /m
A*-APF(1)	1.14	2838	241	415.5
A*-APF(2)	0.79	2481	276	389.4
A*-APF	0.53	1912	184	317.8

Table 6. Table of performance comparison with existing improved A*-APF algorithms.

Unlike the previous general comparisons, this experiment was performed in a large-scale simulation environment specifically designed to mimic a dense urban setting.

The simulation space was defined as 200*20*120 m, representing a typical urban low-altitude sector. To simulate a challenging flight environment populated with dense skyscrapers and high-rise buildings, the obstacle density was set to 35%.

We evaluated the performance of six distinct variants to analyze the progressive impact of the Improved A*, Improved APF, Hybrid Fusion Strategy, Node Expansion Mechanism, Multi-phase Trajectory Modeling, and B-Spline Smoothing. The variants are defined as follows:

Metric	M1	M2	M3	M4	M5	M6
Search Time (s)	0.53	0.63	0.65	0.81	2.45	3.58
Path Length (m)	304	335	398	414	495	457
Number of Valid Nodes	211	247	272	557	Null	769
Success Rate	100%	100%	100%	100%	100%	100%

Table 7. Step-by-step ablation study results.

Number	Position coordinates /m	Three-dimensional Size /m
1	(244, 231, 53)	(14, 11, 53)
2	(340, 333, 55)	(15, 11, 55)

Table 8. Unexpected obstacle information.

M1 (Proposed Full Method): The complete framework integrating Improved A*, Adaptive APF, Multi-phase Modeling, Node Expansion Mechanism, and B-Spline Smoothing.

M2: Removes B-spline Smoothing from M6. The trajectory remains piecewise linear, testing the impact of smoothing on path quality.

M3: Further removes the Multi-phase Trajectory Modeling. The search treats the mission as a single continuous path without phase-specific sub-goals or altitude constraints.

M4: Further removes the Node Expansion Mechanism. Strategies such as Safety Zone Division and Dynamic Step Size are disabled.

M5: Relies solely on the Improved APF for reactive planning without global search guidance.

M6: The algorithm is reduced to Improved A* Only, lacking potential field guidance.

The statistical results, averaged over 50 independent trials, are summarized in Table 7.

It is noteworthy that in Table 7, all variants—including the standalone APF (M5) and A* (M6)—recorded a 100% success rate. This result requires contextual clarification to ensure the credibility of the ablation study. The intuition that APF algorithms are highly prone to local minima is generally accurate for 2D planning. However, in our 3D voxelized simulation, the UAV possesses an extra degree of freedom in the vertical dimension (Z-axis). The standalone APF (M5) achieved a 100% completion rate because, instead of getting permanently trapped in planar dead-ends, it bypassed dense obstacle clusters by executing extreme altitude changes and exploiting 3D spatial gaps.

However, reaching the destination in simulation does not equate to practical feasibility. While M5 technically survived the dense environment without collisions, it exhibited severe oscillatory behavior, unstructured vertical maneuvers, and produced an impractically long flight path (495m compared to the proposed M1’s 304m). Such a highly inefficient and oscillating trajectory is unacceptable in real-world UAV operations due to energy consumption and dynamic limits. Similarly, the standalone A* (M6) successfully found a path, but at the cost of catastrophic computational overhead (3.58s vs 0.53s) and massive redundant node expansions.

The removal of B-spline smoothing (M2) and Multi-phase Modeling (M3) further proved that without these modules, the algorithm degrades into generating suboptimal, non-smooth paths that explore irrelevant altitude layers. In summary, the ablation study confirms that while standalone algorithms might technically ‘survive’ a 3D environment via extreme detours or exhaustive search, only the fully integrated A*-APF method (M1) successfully couples this survival with the stringent computational efficiency and high-quality path characteristics required for practical urban UAV operations. (Note: The ‘Number of Valid Nodes’ metric is not applicable to M5, as it operates on continuous force vectors rather than discrete graph search.

Sudden obstacle simulation

To further validate the effectiveness of the proposed integrated algorithm and its adaptability to urban low-altitude environments, as well as to address potential sudden local dynamic threats during flight, sudden obstacles were incorporated into the static obstacle space while adhering to the UAV’s performance constraints. These sudden obstacles, represented by dark red buildings, are detailed in Table 8. A comparative simulation between the A*-APF algorithm and the SAS algorithm was conducted to evaluate the dynamic obstacle avoidance performance of the proposed method. The simulation results are presented in Fig. 13.

As shown in Fig. 16, if a temporary obstacle appears on the optimal trajectory of the UAV, the SAS algorithm fails to avoid it effectively, leading to a failure in the UAV path planning mission. When applied to UAV trajectory planning, the A*-APF algorithm successfully navigates through pre-existing static threats along the desired reference trajectory until encountering a sudden obstacle. Under these conditions, the artificial potential field method is activated to perform local trajectory replanning, enabling effective avoidance of the unexpected threat. After obstacle avoidance, the UAV seamlessly returns to the reference trajectory. The comparative simulation results further demonstrate the effectiveness of the proposed integrated algorithm and its suitability for urban low-altitude environments.

The flight safety evaluation metric for UAVs in response to sudden obstacles is defined as follows:

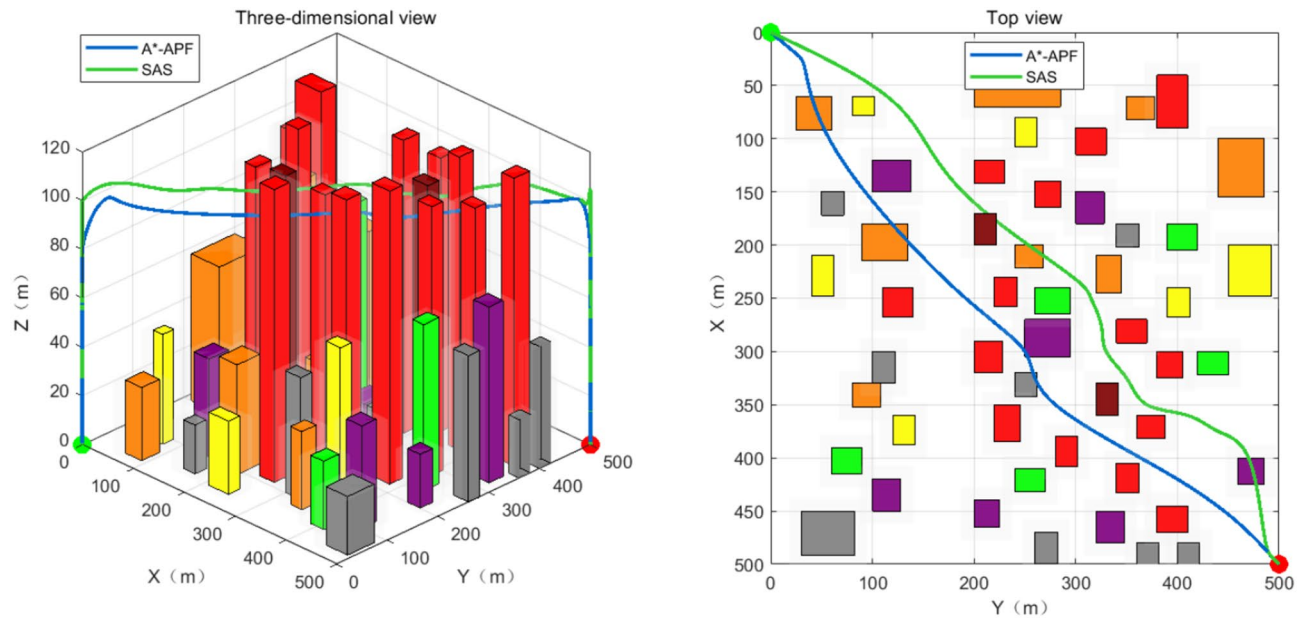


Fig. 16. Complex low-altitude airspace trajectory planning diagram.

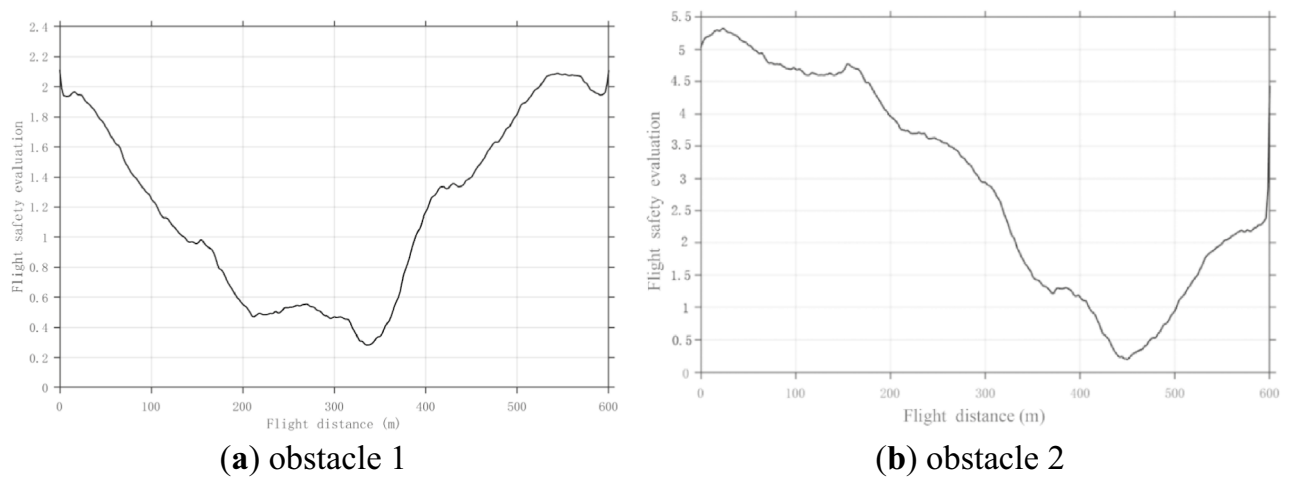


Fig. 17. Comparison diagram of flight safety evaluation metrics.

$$F_{\text{index}} = \left(\frac{x_i - x_o}{a} \right)^2 + \left(\frac{y_i - y_o}{b} \right)^2 + \left(\frac{z_i - z_o}{c} \right)^2 - 1 \quad (67)$$

In Eq. (58), (x_p, y_p, z_p) denotes the real-time coordinates of the UAV; (x_o, y_o, z_o) represents the coordinates of the obstacle; (l_x, l_y, l_z) indicates the length, width, and height of the obstacle; and F_{index} refers to the flight safety evaluation metric. A value of F_{index} greater than 0 indicates that the UAV maintains a safe distance from the sudden obstacle, while a value less than or equal to 0 signifies a failure in collision avoidance.

As shown in Fig. 17, the flight safety evaluation metrics were assessed in response to two emergent obstacles introduced into the three-dimensional airspace. The results indicate that the corresponding flight safety metrics consistently remained greater than zero. This demonstrates that activating the improved Artificial Potential Field (APF) algorithm when the UAV encounters obstacles during flight effectively compensates for the limitations of the A* algorithm in obstacle avoidance, thereby enabling successful trajectory planning in complex three-dimensional environments.

Conclusions

This paper addresses challenges such as dense obstacles, complex mission requirements, and poor adaptability of traditional algorithms in multi-rotor UAV path planning within urban low-altitude environments. A multi-

phase trajectory planning method is proposed, which integrates an improved A* algorithm with an adaptive artificial potential field (A*-APF). By employing 3D airspace modeling, phased path progression, dynamic step size control, and B-spline trajectory smoothing strategies, a comprehensive UAV trajectory planning system covering the entire flight process—from take-off and cruise to landing—has been constructed.

Through multiple sets of comparative simulations, ablation studies, and analyses in both 2D and 3D scenarios, the following conclusions are drawn:

- (1) Full Mission Profile Control: The introduction of 3D voxel-based modeling and a flight-phase progression mechanism enables integrated path control throughout the entire mission profile. This approach enhances the executability of the trajectory for real-world operations by ensuring smooth transitions between take-off, cruise, and landing phases.
- (2) Synergy of A and APF: The integration of the improved A* algorithm with the adaptive artificial potential field method fully leverages the global guidance capability of A* and the local obstacle avoidance ability of APF. This synergy significantly enhances search efficiency and avoidance performance while ensuring operational safety.
- (3) Efficiency via Node Expansion: The constructed multi-strategy node expansion mechanism demonstrates high flexibility in guiding expansion directions and performing kinematic filtering. It effectively reduces redundant node expansion, thereby improving both the computational efficiency and the quality of path planning.
- (4) Validation of Structural Innovation: The rigorous ablation study and comparative analysis quantitatively confirm that the performance gains are driven by structural innovations rather than mere parameter tuning. Specifically, the Multi-phase Modeling is identified as the primary contributor to global efficiency by pruning the search space, while the Node Expansion Mechanism ensures local safety. Furthermore, the proposed method demonstrates superior planning speed and trajectory optimality compared to existing state-of-the-art hybrid algorithms^{15,16}.

In summary, the proposed A*-APF integrated multi-phase trajectory planning method provides strong theoretical support and technical reference for the intelligent and autonomous operation of multi-rotor UAVs in urban low-altitude airspace, demonstrating significant potential for practical application.

Data availability

The datasets generated during and/or analysed during the current study are not publicly available due to the proprietary nature of the simulation models and core algorithm parameters but are available from the corresponding author on reasonable request.

Received: 18 September 2025; Accepted: 17 March 2026

Published online: 20 April 2026

References

1. Guban, G. & Haque, A. Path planning for autonomous drones: Challenges and future directions. *Drones* **7**(3), 169 (2023).
2. Debnath, D., Vanegas, F., Sandino, J., Hawary, A. F. & Gonzalez, F. A review of UAV path-planning algorithms and obstacle avoidance methods for remote sensing applications. *Remote Sens.* **16**(21), 4019 (2024).
3. Wang, Y.H., Zhang, W. J., Zhou, T. X. Path planning of surface unmanned craft based on variable step-length sparse a-star algorithm. In *Proceedings of the 2021 33rd Chinese Control and Decision Conference (CCDC)* (IEEE, Kunming, 2021) 1917–1920.
4. Gan, Y. et al. Research on robot motion planning based on RRT algorithm with non-holonomic constraints. *Neural Process. Lett.* **53**(4), 3011–3029 (2021).
5. Shen, K. Y. et al. Mobile robot path planning based on improved A* algorithm. *Comput. Appl. Res.* **40**(1), 75–79 (2023).
6. Liu, L. S. et al. Path planning for smart car based on dijkstra algorithm and dynamic window approach. *Wirel. Commun. Mob. Comput.* <https://doi.org/10.1155/2021/8881684> (2021).
7. Huang, S. Z. et al. UAV path planning based on improved genetic algorithm. *Comput. Appl.* **41**(2), 390–397 (2021).
8. Zhang, H. et al. Mobile robot path planning based on improved two-layer ant colony algorithm. *Control Decis.* **37**(2), 303–313 (2022).
9. Huang, Z. F. & Liu, Y. H. Solving path planning problem based on fourth-order Bezier curve and improved lion swarm optimization algorithm. *Inf. Control* **52**(2), 176–189 (2023).
10. Chen, P. et al. UAV trajectory planning method based on local soft constraint optimization. *J. South China Univ. Technol. (Nat. Sci. Ed.)* **50**(6), 27–36 (2022).
11. Zhang, X., Ma, Y. & Liu, Z. Improved A* algorithm for UAV global path planning in 3D environment. *Sensors* **22**(11), 4260 (2022).
12. Khatib, O. Real-time obstacle avoidance for manipulators and mobile robots. *Int. J. Robot. Res.* **5**(1), 90–98 (1986).
13. Hao, G., Lv, Q., Huang, Z., Zhao, H. & Chen, W. UAV path planning based on improved artificial potential field method. *Aerospace* **10**(6), 562 (2023).
14. Xue, T., Cao, Y., Zhao, Y., Ai, J. & Dong, Y. Hybrid A*-based valley path planning algorithm for aircraft. *Aerospace* **11**(7), 516 (2024).
15. Tang, Y. Y., Zheng, E. H. & Qiu, X. 3D trajectory planning for UAV based on optimized bidirectional A* and artificial potential field method. *J. Air Force Eng. Univ.* **25**(5), 69–75 (2024).
16. Guo, L., Zhao, C., Li, J., Yan, Q., Li, W., Chen, P. UAV path planning based on improved A* and artificial potential field algorithm. In *2024 36th Chinese Control and Decision Conference (CCDC)* (IEEE, Xi'an, 2024) 5471–5476.
17. Lin, J. et al. Unmanned aerial vehicle-enabled aerial radio environment map construction: A multi-stage approach to data sampling and path planning. *Drones* **9**(2), 81 (2025).
18. Li, Q., Ma, Q. & Weng, X. Dynamic path planning for mobile robots based on artificial potential field enhanced improved multiobjective snake optimization. *J. Field Robot.* **41**(6), 1843–1863 (2024).
19. Ding, W., Zhang, L., Chen, J. & Shen, S. Safe trajectory generation for complex urban environments using Spatio-temporal semantic corridor. *IEEE Robot. Autom. Lett.* **4**(3), 2997–3004 (2019).
20. Guo, Z. Efficient path planning method based on A-STAR. In *Journal of Physics: Conference Series* Vol. 2649(1) (IOP Publishing, 2023) 012026.

21. Zhang, D., Chen, C., Zhang, G. AGV path planning based on improved A-star algorithm. In *2024 IEEE 7th advanced information technology, electronic and automation control conference* Vol. 7 (IAEAC) (IEEE, 2024) 1590-1595.
22. Meng, X. et al. Obstacle avoidance path planning for unmanned aerial vehicle in workshops based on parameter-optimized artificial potential field A* algorithm. *Machines* **13**(10), 967 (2025).

Author contributions

The authors confirm contribution to the paper as follows: Study conception and design, G.W. and L.L.L.; methodology, P.D.Y.; software, L.L.L.; validation, G.W., L.L.L. and P.D.Y.; formal analysis, G.W.; data curation, L.L.L.; writing—original draft preparation, P.D.Y.; writing—review and editing, G.W. All authors have read and agreed to the published version of the manuscript.

Funding

This research is supported by the” Key Technologies for Integrated Operations of Manned and Unmanned Aircraft”project, The National Key Research and Development Program of China (2022YFB4300904).

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to W.G.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher’s note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026